

Dokumentacja techniczna projektu

**Dokumentacja projektu grupowego
ID-346 - Analiza bio-sygnałów do identyfikacji osób**

Mikołaj Kuźmich 198291, Jakub Dąbrowski 197629, Mikołaj Jaworski 197636

Spis treści

1	Konwencje	4
1.1	Słownik pojęć:	4
1.2	Metoda pomiarowa	5
1.3	Wykorzystane technologie	5
2	Przetwarzanie danych	7
2.1	Obiekt pomiaru	7
2.1.1	Charakterystyka surowych danych	7
2.2	Etapy przetwarzania danych	8
2.2.1	Obsługa danych wejściowych i wstępne przetwarzanie	8
2.2.2	Oddzielanie oddechów	9
2.2.3	Wykrywanie anomalii	9
2.2.4	Podział danych na segmenty i resampling	11
2.2.5	Ekstrakcja cech	12
2.3	Wizualizacja danych	13
2.3.1	SVM - Support Vector Machine	14
2.4	Tworzenie profili oddechowych osób	16
2.4.1	Metodyka wyznaczania profilu	16
2.4.2	Zastosowanie profili	17
3	Kod źródłowy - toolkit	19
3.1	Przeływ sterowania w głównym potoku przetwarzania <code>data_processing_pipeline</code>	19
3.1.1	Klasy wykorzystywane w potoku przetwarzania danych	19
3.2	Architektura systemu	20
3.3	Kluczowe funkcje i przeływ danych wewnątrz skryptu <code>initial_data_processing</code>	20
3.4	Kluczowe funkcje i przeływ danych wewnątrz skryptu <code>extract_features</code>	21
3.5	Kluczowe funkcje i przeływ danych wewnątrz skryptu <code>visualize_data</code>	21
3.6	Kluczowe funkcje i przeływ danych wewnątrz skryptu <code>create_data_profiles</code>	21
3.7	Utworzenie połączenia oraz obsługa zapytań do API - <code>DatabaseHandler</code>	22
3.8	Interakcja z archiwami pomiarów	22
3.9	BRICStoolkit	23
4	Serwer BRICS	24
4.1	BRICS DB	24
4.2	API	25
4.2.1	Punkty końcowe API	25
4.2.2	Szczegółowa realizacja punktów końcowych	26
4.2.3	Redis	27
4.2.4	Celery Worker	27
4.2.5	Celery Beat	28
4.3	Zabezpieczenia	28
4.3.1	Konsola zdalna	29
4.3.2	API	29
4.3.3	Docker	29
5	Płytki pośrednicząca w komunikacji	30
5.1	Architektura płytki	30
5.2	Wielowątkowa aplikacja pośrednicząca	30

5.3	Serwer Bluetooth	31
5.3.1	Działanie serwera Bluetooth LE	31
5.4	Sygnalizowanie stanu płytki GPIO	31
5.5	Przykładowy scenariusz użycia	32
6	Model BRICS	33
6.1	Organizacja repozytorium	33
6.2	Klasyfikatory	34
6.2.1	Klasyfikator danych oczyszczonych	34
6.2.2	Klasyfikator wybranych cech	36
6.3	Dane	39
6.3.1	Pobieranie danych	39
6.3.2	Parsowanie cech	40
6.4	Model	41
6.4.1	Klasy będące pojemnikami na dane	41
6.4.2	Model nauczania maszynowego	42
6.4.3	Założenia treningowe	43
6.4.4	Przebieg treningu	45
6.4.5	Wyniki	46
6.5	Aplikacja	47
	Lista rysunków	50
	Lista tabel	51
	Bibliografia	52

1 Konwencje

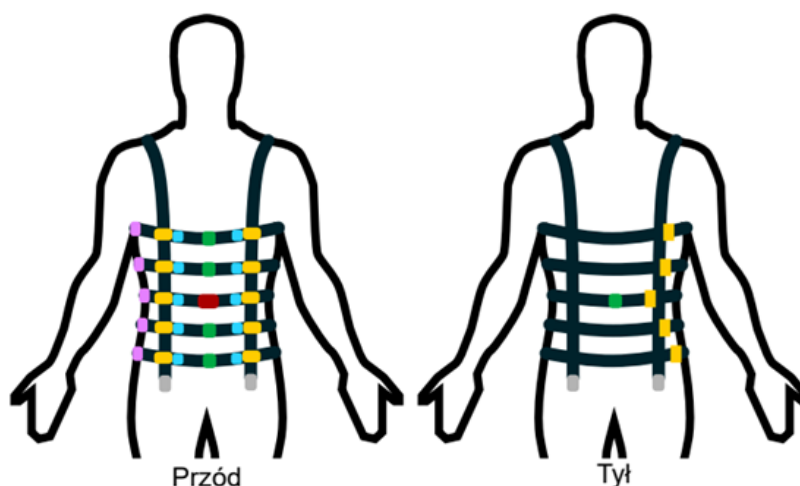
1.1 Słownik pojęć:

- **BRV (Breathing Rhythm Vest)** - kamizelka pomiarowa wyposażona w tensometry, akcelerometry i żyroskopy, służąca do pomiaru rytmu oddechowego u człowieka. Główny instrument pomiarowy, wykonany w ramach pracy magisterskiej innej grupy studentów WETI
- **ADC (Analog-to-Digital Converter)** - przetwornik analogowo-cyfrowy, urządzenie elektroniczne służące do konwersji sygnałów analogowych na cyfrowe. Tu jest to część kamizelki BRV, która przetwarza sygnały z tensometrów na dane cyfrowe do dalszej analizy.
- **Pomiar/badanie** - całość pliku z danymi zebranymi podczas jednego badania, trwającego określonej ilości czasu (np. 10 minut).
- **Próbka** - pojedynczy znacznik czasu z przypisanymi wartościami sygnału ADC z czujników kamizelki BRV.
- **Segment** - wyodrębniony fragment pomiaru o określonej długości czasowej (np. 2 minuty), wykorzystywany do analizy i ekstrakcji cech. Zawiera wiele próbek.
- **Cecha** - wyodrębniona wartość liczbową lub wektor wartości liczbowych, będąca numeryczną reprezentacją pewnego aspektu sygnału oddechowego w danym segmencie.
- **Toolkit** - skrócona wersja określenia BRICStoolkit. Jest to zestaw narzędzi (klas, funkcji oraz skryptów) używanych w wielu miejscach systemu BRICS. Został stworzony, aby uniknąć powielania kodu oraz stanowi niezależną bibliotekę w *Pythonie*.

1.2 Metoda pomiarowa

Dla kamizelki BRV wyznaczono następujący sposób dostosowania poziomu pasów:

- pas 1 - pod pachami
- pas 2 - na linii sutków
- pas 3 - na poziomie najniższego z żeber
- pas 4 - na poziomie pępka
- pas 5 - na poziomie pasa



Rysunek 1.1: Graficzna reprezentacja sposobu noszenia kamizelki pomiarowej BRV.

Obecnie wszystkie wykonane pomiary odbywały się w pozycji siedzącej i w stanie spoczynku. Dopuszczalne były jedynie niewielkie ruchy ciała, takie jak poprawienie pozycji siedzącej czy zmiana ułożenia rąk. Dodatkowo dopuszczaliśmy następujące czynności:

- picie wody,
- używanie telefonu komórkowego
- czytanie książki
- korzystanie z komputera - tu należy zaznaczyć, że było to korzystanie bez znaczących ruchów ciała i w sposób niewzbudzający silnych emocji (może to powodować zmiany naturalnego rytmu oddechowego).

Całe badanie zazwyczaj trwało około 15 minut, ale wykonaliśmy też kilka dłuższych pomiarów. Najdłuższy wykonany pomiar trwał 60 minut. Całkowicie udało się zebrać około 5 godzin materiału pomiarowego od 3 różnych osób. Powiększenie tego zbioru (zarówno pod względem czasu pomiarowego, jak i liczby osób) jest priorytetem na przyszłość.

1.3 Wykorzystane technologie

W projekcie zostały wykorzystane następujące technologie:

- Język programowania Python w wersji 3.12
- Oprogramowanie LaTeX do tworzenia dokumentacji technicznej

- Biblioteka NumPy do obliczeń numerycznych
- Biblioteka SciPy do analizy sygnałów
- Biblioteka Matplotlib do wizualizacji danych
- Biblioteka Seaborn do wizualizacji danych
- Biblioteka SciKit-learn do uczenia maszynowego
- Biblioteka Pathlib do operacji na ścieżkach plików
- Baza danych MongoDB do przechowywania danych pomiarowych
- Framework FastAPI do stworzenia REST API dla bazy danych
- Serwer WWW Uvicorn
- Zbiór narzędzi do wytwarzania i uruchamiania aplikacji skonteneryzowanych Docker

2 Przetwarzanie danych

2.1 Obiekt pomiaru

Obiektem pomiaru jest sygnał oddechowy rejestrowany przez system kamizelki sensorycznej. Pomiar opiera się na analizie zmian obwodu klatki piersiowej i brzucha użytkownika podczas cyklu oddechowego.

System wykorzystuje dwa główne typy sensorów:

- **Tensometry (ADC):** Pięć pasów tensometrycznych rozmieszczonych na różnych wysokościach tułowia. Rejestrują one siłę naciągu pasów, która zmienia się wraz z nabieraniem i wypuszczaniem powietrza.
- **Jednostki inercyjne (IMU):** Zestaw akcelerometrów i żyroskopów monitorujących orientację przestrzenną oraz ruchy klatki piersiowej. Na chwilę obecną dane z tych sensorów nie są wykorzystywane.

2.1.1 Charakterystyka surowych danych

Dane przesyłane z urządzenia mają formę strumienia obiektów JSON. Każda próbka zawiera timestamp oraz surowe odczyty binarne z sensorów.

Poniżej przedstawiono strukturę kluczowych pól surowej próbki:

Tabela 2.1: Opis składowych danych surowych.

Klucz sygnału	Typ	Opis fizyczny
hrs, min, sec, ms	czas	Czas rejestracji próbki od uruchomienia urządzenia.
adc1...adc5	24-bit	Trzy 8-bitowe rejestry (np. 23_to_16) tworzące naciąg pasa zapisany w formacie U2
imu1...imu5_acc	m/s ²	Składowe przyspieszenia liniowego w osiach X, Y, Z dla 5 punktów kamizelki.
imu1...imu5_gyr	deg/s	Prędkość kątowna (orientacja) klatki piersiowej i pleców.
output_status	bool	Flaga diagnostyczna określająca poprawność pracy danego sensora.

Poniżej przedstawiono przykładowy format surowej próbki:

```
"hrs": 0, "min": 0, "sec": 13, "ms": 457,  
"adc1_output_23_to_16": 240, "adc1_output_15_to_8": 8,  
"adc1_output_7_to_0": 165, "adc2_output_23_to_16": 255,  
"adc2_output_15_to_8": 36, "adc2_output_7_to_0": 15,  
"adc3_output_23_to_16": 243, "adc3_output_15_to_8": 99,  
"adc3_output_7_to_0": 199, "adc4_output_23_to_16": 253,  
"adc4_output_15_to_8": 161, "adc4_output_7_to_0": 55,  
"adc5_output_23_to_16": 238, "adc5_output_15_to_8": 46,  
"adc5_output_7_to_0": 1, "imu1_output_acc_x": 2384,  
"imu1_output_acc_y": 237, "imu1_output_acc_z": 16526,  
"imu1_output_gyr_x": -12, "imu1_output_gyr_y": -11,
```

```

"imu1_output_gyr_z": 17, "imu2_output_acc_x": 3021,
"imu2_output_acc_y": 888, "imu2_output_acc_z": 16327,
"imu2_output_gyr_x": -1, "imu2_output_gyr_y": -31,
"imu2_output_gyr_z": 17, "imu3front_output_acc_x": 5654,
"imu3front_output_acc_y": 866, "imu3front_output_acc_z": 15351,
"imu3front_output_gyr_x": -2, "imu3front_output_gyr_y": 7,
"imu3front_output_gyr_z": -5, "imu3back_output_acc_x": -831,
"imu3back_output_acc_y": -2094, "imu3back_output_acc_z": 16238,
"imu3back_output_gyr_x": 2, "imu3back_output_gyr_y": 9,
"imu3back_output_gyr_z": -9, "imu4_output_acc_x": 1997,
"imu4_output_acc_y": 655, "imu4_output_acc_z": 15972,
"imu4_output_gyr_x": -5, "imu4_output_gyr_y": -4,
"imu4_output_gyr_z": -2, "imu5_output_acc_x": -768,
"imu5_output_acc_y": 437, "imu5_output_acc_z": 16111,
"imu5_output_gyr_x": -3, "imu5_output_gyr_y": 3,
"imu5_output_gyr_z": 7, "output_status_adc1": true,
"output_status_adc2": true, "output_status_adc3": true,
"output_status_adc4": true, "output_status_adc5": true,
"output_status_imu1": true, "output_status_imu2": true,
"output_status_imu3front": true, "output_status_imu3back": true,
"output_status_imu4": true, "output_status_imu5": true,
"output_status_padding": 0

```

2.2 Etapy przetwarzania danych

2.2.1 Obsługa danych wejściowych i wstępne przetwarzanie

Pierwszym etapem przetwarzania danych jest ich wczytanie oraz odpowiednie sformatowanie. Dane, zgodnie z opisem w punkcie 2.1, są przechowywane w plikach `.jsonl`. Każda linia reprezentuje pojedynczą próbkę zarejestrowaną przez kamizelkę pomiarową.

Wczytywanie odbywa się sekwencyjnie. Każda linia zawiera surowe dane czasowe (godziny, minuty, sekundy, milisekundy), 24-bitowe wartości z pięciu przetworników ADC (podzielone na trzy 8-bitowe składowe) oraz dane z czujników inercyjnych IMU.

W celu ułatwienia dalszej obróbki, każda z tych linii jest parsowana do wewnętrznej struktury danych. Proces ten obejmuje połączenie trzech 8-bitowych rejestrów każdego z pięciu kanałów ADC w jedną wartość liczbową, która następnie jest przeliczana na napięcie (wolt) zgodnie ze specyfikacją zastosowanych tensometrów.

Struktura danych wynikowych

Po wstępnym przetworzeniu dane są normalizowane i zapisywane w formie uproszczonych obiektów JSON, które stanowią podstawową jednostkę danych w dalszych etapach projektu. Przykładowy format przetworzonego segmentu (np. 2-minutowego) wygląda następująco:

```

{
  "timestamp": 0,
  "adc_outputs": [
    0.0002464437,
    0.0003881085,
    0.0004560553,
    -1.75746999e-05,
    0.0002706136
  ]
}

```


Tu warto dodać, że dla większości obliczeń wykonywanych w dalszych etapach takich jako wyciąganie cech, separacja oddechów czy wykrywanie anomalii pracujemy na sygnale dla środkowego ADC (ADC numer 3), eksperymentalnie ustaliliśmy, że jest on najbardziej miarodajny w kontekście analizy oddechu, a praca z jednym czujnikiem znacznie upraszcza cały proces.

2.2.2 Oddzielanie oddechów

Celem tego etapu jest wstępne podzielenie sygnału na mniejsze części reprezentujące kolejne oddechy w analizowanym pomiarze. Realizowane jest to poprzez wyznaczenie wszystkich maksymów i minimów lokalnych oraz zapisanie ich w programie tak, by przygotować je do dalszej obróbki.

- **Dane wejściowe:**

Zbiór wartości odstawiania od średniej wartości sygnału na danym pasie, dla każdego pasa w kamizelce pomiarowej. Każda z tych wartości przypisany ma odpowiedni znacznik czasu od rozpoczęcia pomiaru, w którym pobrano próbkę.

- **Dane wyjściowe:**

Zbiór punktów reprezentujących wszystkie maksyma i minima lokalne sygnału wraz z przypisanymi im wartościami znaczników czasu.

2.2.3 Wykrywanie anomalii

Kolejnym kluczowym etapem jest identyfikacja oraz eliminacja anomalii (ang. *outlier detection*). Sygnał z czujników tensometrycznych w kamizelce jest podatny na zakłócenia wynikające z nagłych ruchów użytkownika, poprawiania kamizelki czy chwilowych błędów transmisji danych. W ten sposób mogą pojawić się oddechy o nietypowych wartościach, które mogą negatywnie wpłynąć na jakość analizy i klasyfikacji.

Proces ich wykrywania i usuwania opiera się na analizie statystycznej zbioru wyznaczonych wcześniej ekstremów (maksimów i minimów) oraz odległości czasowych między nimi.

- **Kryteria wykrywania anomalii:**

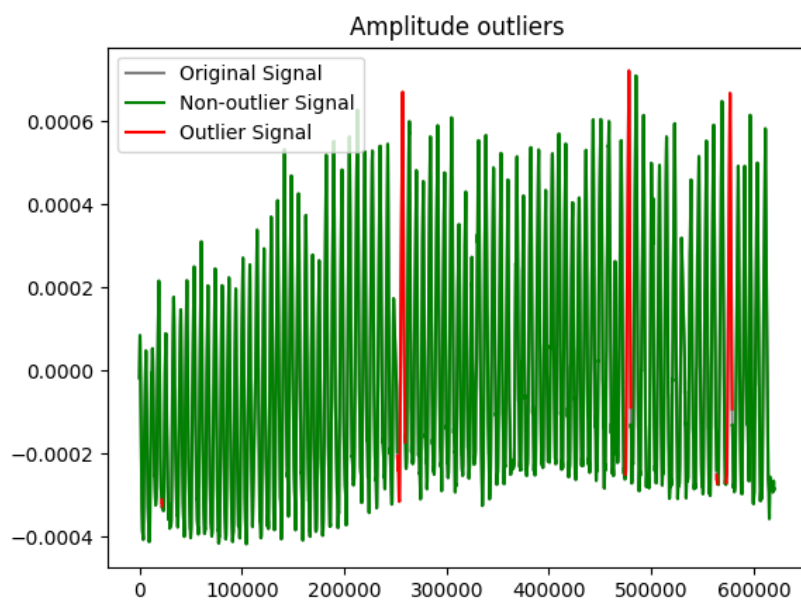
- **Amplituda:** Odrzucane są oddechy o wartościach należących do górnych i dolnych kwartyli rozkładu amplitud. Obecnie odrzucamy około jeden procent oddechów z każdej strony rozkładu.
- **Częstotliwość:** Odrzucane są oddechy o zbyt krótkich oraz długich odległościach między wykrytymi wcześniej maksymami. Obecnie odrzucamy około jeden procent oddechów z każdej strony rozkładu.

- **Dane wejściowe:**

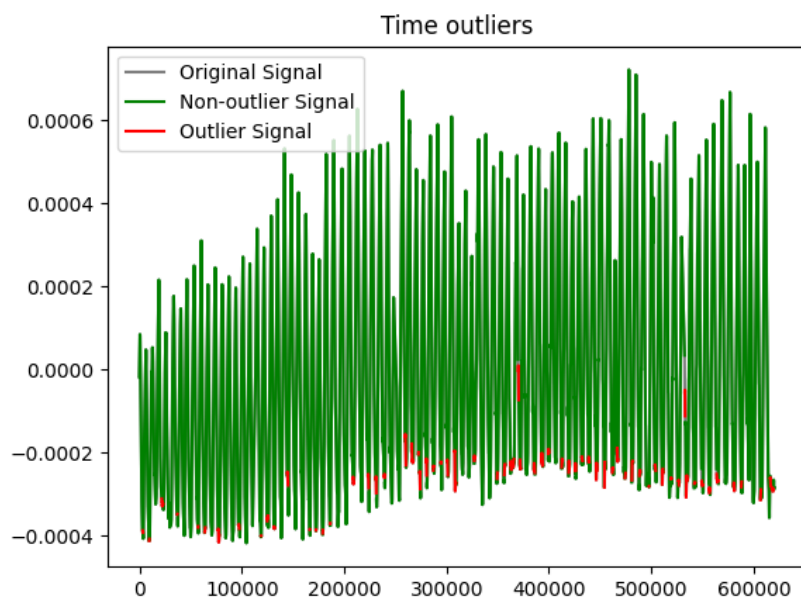
Zbiór par (znacznik czasu, wartość) dla zidentyfikowanych maksimów i minimów lokalnych z poprzedniego etapu oraz znormalizowany sygnał oddechowy.

- **Dane wyjściowe:**

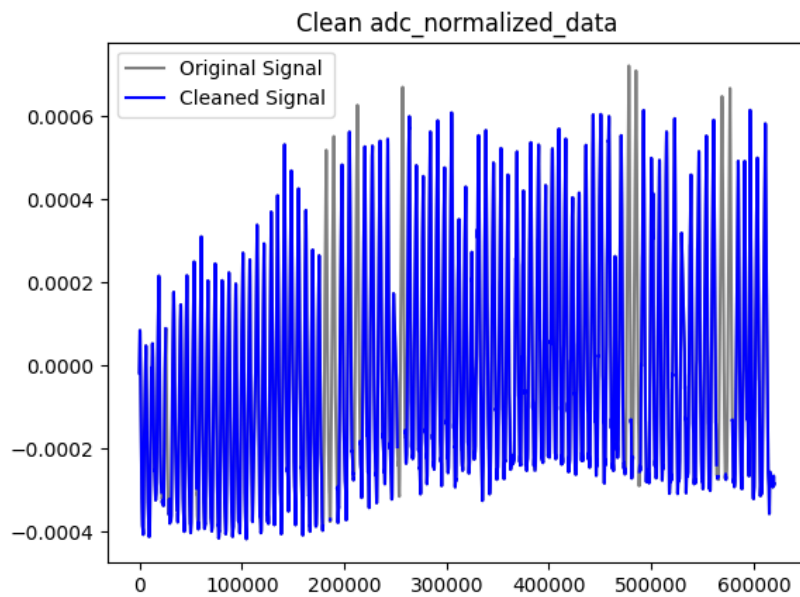
Przefiltrowany zbiór punktów (zmodyfikowany sygnał oddechowy), o odpowiednio przesuniętych wartościach znaczników czasu.



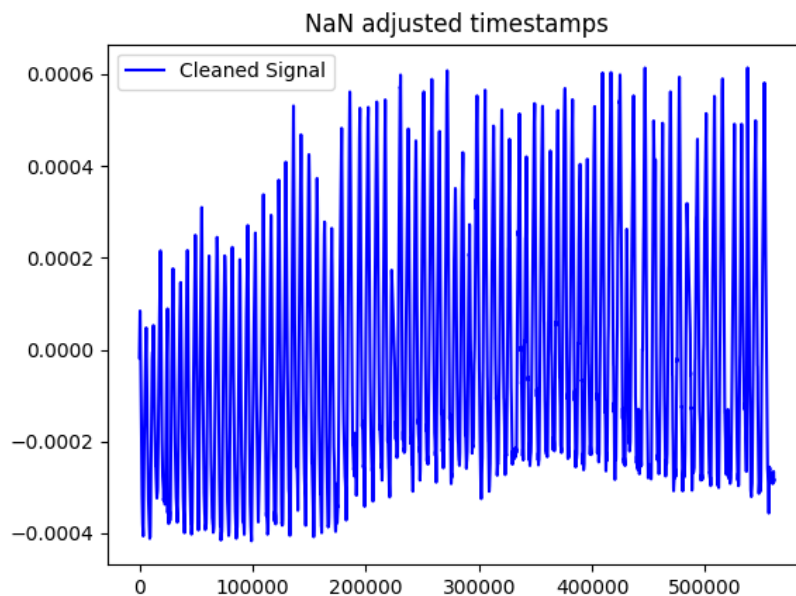
Rysunek 2.1: Wykryte anomalie na podstawie amplitudy dla przykładowego sygnału.



Rysunek 2.2: Wykryte anomalie na podstawie Częstotliwości dla przykładowego sygnału.



Rysunek 2.3: Wykryte anomalie po usunięciu artefaktów dla przykładowego sygnału.



Rysunek 2.4: Odtworzony sygnał po usunięciu anomalii dla przykładowego sygnału.

2.2.4 Podział danych na segmenty i resampling

Ostatnim krokiem przygotowania danych przed ekstrakcją cech jest podział przefiltrowanego sygnału na okna czasowe (segmenty) o stałej długości dwóch minut oraz wykonanie resamplingu.

Najpierw sygnał jest resamplowany do za pomocą interpolacji liniowej (funkcja `interp1d` z biblioteki SciPy) do Częstotliwości 10 Hz, a następnie dzielony na segmenty. Dzięki temu każdy segment zawiera około 1200 próbek. Sam algorytm podziału na segmenty jest raczej prosty - rozpoczynamy od początku sygnału i zapisujemy do kolejnych plików `.jsonl` kolejne próbki danych zawierające się w danym dwuminutowym segmencie.

Tu warto wspomnieć, że segmenty nie muszą być dwuminutowe, a jedynie obecnie, po konsultacjach oraz analizie, zdecydowaliśmy się na takich pracować. Głównym powodem było zachowanie równowagi pomiędzy długością segmentu - tak aby jeden pomiar oferował wiele segmentów do analizy, a ilością danych w pojedynczym segmencie - musi być to miarodajna próbka ludzkiego oddechu, z której da się pozyskać zestaw określonych cech. Dla innych zastosowań projektu np. identyfikacji osoby w czasie rzeczywistym długość segmentów ulegnie zmianie do 10 sekund.

2.2.5 Ekstrakcja cech

Po podzieleniu danych na segmenty, a więc ostatecznym przygotowaniu ich do analizy, należy rozpocząć proces pozyskiwania informacji potrzebnych do prawidłowego działania klasyfikatora. Sama sieć neuronowa nie potrzebuje zdefiniowanych cech sygnału, jednak w celach badawczych, zdecydowano się na ich ekstrakcję ze względu na możliwość sprawdzenia różnic we wzorcach oddechowych objętych analizą osób oraz porównanie efektywności identyfikacji w oparciu o zbiór cech z klasyfikacją sygnałów czasowych (ang. *outlier detection*, TSC).

- **Dane wejściowe:**

Zbiór plików zawierających podzielone na segmenty dane z pomiarów po wszystkich przekształceniach, w których nazwa wyznacza etykietę. Nazwa pliku zawiera typ aktywności oraz identyfikator osoby

- **Dane wyjściowe:**

Plik wyjściowy `extracted_features.jsonl` w formacie `.jsonl`, zawierający wektory cech wyznaczone dla kolejnych segmentów danych pomiarowych.

Cechy składające się na wektor dla segmentu:

- `bpm` - liczba oddechów na minutę
- `avg_breath_depth` - wyznaczone poprzez uśrednienie wartości szczytów oddechów w całym segmencie dla środkowego pasa
- `avg_breath_depth_std_dev` - odchylenie standardowe sygnału, jako głębokość oddechu dla środkowego pasa
- `phase_avg_values1-4` - uśrednione wartości długości poszczególnych faz oddychania
- `breath_shape1-4` - uśrednione wartości współczynników wielomianu aproksymowanego na wszystkich oddechach
- `breath_length_variability` - jest to średnia kwadratowa kolejnych różnic w długości oddechów (RMSSD)
- `breath_amplitude_variability` - jest to współczynnik wariancji obliczony na podstawie wszystkich amplitud oddechów i ich wariancji (CV)
- `belt_share1-5` - udział poszczególnych pasów w oddychaniu, reprezentowany przez 5 wartości, po jednej dla każdego pasa. Jest on wyznaczany poprzez zsumowanie wartości głębokości oddechu dla każdego pasa, tworzy to mianownik ułamka, który liczony jest osobno dla każdego tensometru, gdzie licznikiem jest głębokość oddechu dla tego konkretnego sensora
- `belt_share_std1-5` - wyznaczany jest on analogicznie do powyższego, korzysta jednak z wyżej wspomnianych odchyłeń standardowych sygnału, jako głębokości

Ostateczna postać wektora cech:

```
{
  "bpm": 22.26614295364139,
  "avg_breath_depth": 3.900225777541929e-05,
  "avg_breath_depth_std_dev": 0.00013306949022660511,
  "phases_avg_values1": 940.4347826086956,
  "phases_avg_values2": 0.0,
  "phases_avg_values3": 1178.7826086956522,
  "phases_avg_values4": 103.82608695652173,
  "breath_shape1": 9.196209863545878e-08,
  "breath_shape2": -6.662249686393455e-06,
  "breath_shape3": 9.61982727984441e-05,
  "breath_shape4": -0.00042464473377830305,
  "breath_length_variability": 545.1113966728349,
  "breath_amplitude_variability": 1.1727109719885525,
  "belt_share1": 0.3316531281145216,
  "belt_share2": 0.0650033517833078,
  "belt_share3": 0.1339896910044088,
  "belt_share4": 0.2490435239116464,
  "belt_share5": 0.22031030518611552,
  "belt_share_std1": 0.17940369491358457,
  "belt_share_std2": 0.2232564079584426,
  "belt_share_std3": 0.2208655713802452,
  "belt_share_std4": 0.18995479665889553,
  "belt_share_std5": 0.186519529088832
}
```

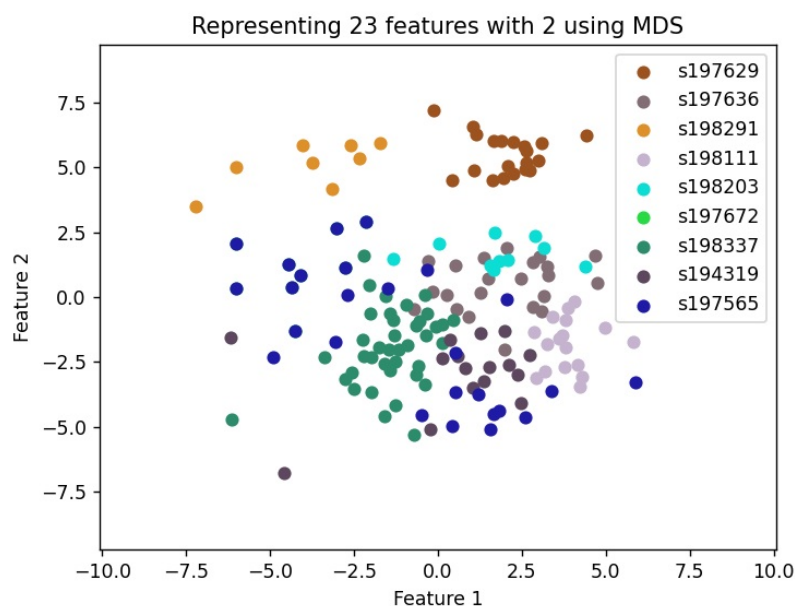
2.3 Wizualizacja danych

Aby zweryfikować poprawność całego procesu przetwarzania należało w odpowiedni sposób zwizualizować dane pozyskane w ramach projektu. Jest to niezbędny etap, gdyż pozwala na ewaluację postępów.

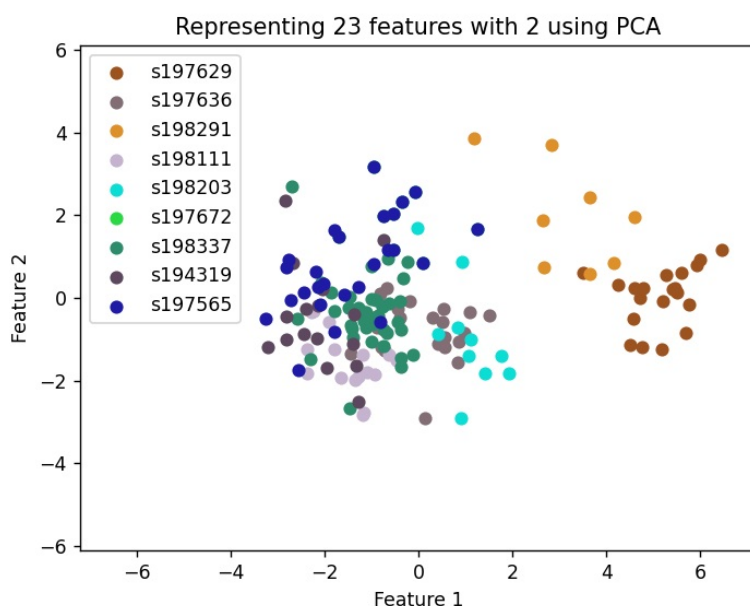
Trzy główne cele etapu to:

1. **Weryfikacja rozróżnialności osób po przetwarzaniu** - Aby móc zweryfikować poprawność procesu przetwarzania, trzeba zwizualizować zebrane dane. Jest to realizowane za pomocą algorytmów redukujących liczbę cech: **MDS** oraz **PCA**. W tym podpunkcie liczba składowych, w celach poglądowych, obniżana jest do dwóch. Dzięki tym wartościom jesteśmy w stanie wyznaczyć kierunek, będący transformacją macierzy danych, który spowoduje największe możliwe zróżnicowanie zbioru danych. Zakładając więc, że różnice we wzorcach oddechowych między różnymi osobami będą stosunkowo dużo większe niż między różnymi momentami dla tej samej osoby, jesteśmy w stanie uzyskać odpowiednią, wizualną separację danych, zgodną z etykietami.
2. **Wyznaczenie najbardziej istotnych cech dla klasyfikatora** - W celu wyżej wspomnianej weryfikacji przetwarzania danych, należało zredukować liczbę wymiarów wektora cech, tak by ten mógł zostać przedstawiony w sposób możliwy do analizy przez człowieka. Zostały więc wyznaczone w ten sposób najbardziej istotne elementy sygnału. Dzięki temu jesteśmy w stanie zredukować ostateczną ich liczbę, co pozytywnie wpływa na ryzyko zbytniego dopasowania oraz pozwala na mniejszy nakład obliczeniowy związany np. ze skorelowanymi zmiennymi.

Warto dodać, że przedstawione tutaj cechy, oznaczone jako **"Feature1"** i **"Feature2"** są kombinacjami liniowymi wszystkich poprzednich cech zgodnie z działaniem obu algorytmów redukcji wymiarowości. Nie są one więc możliwe do bardziej szczegółowego opisu. Dodatkowo, nie wszystkie wektory cech zostały na wykresach przedstawione, gdyż dane zostały obcięte powyżej progu trzech odchyłeń standardowych, aby zmaksymalizować czytelność.



Rysunek 2.5: Cechy po wywołaniu algorytmu MDS przedstawione na dwuwymiarowym wykresie.



Rysunek 2.6: Cechy po wywołaniu algorytmu PCA przedstawione na dwuwymiarowym wykresie.

3. **Kontrola jakości przez prosty klasyfikator** - Ostatnim celem Wizualizacji jest kontrola jakości rozwiązania dzięki klasyfikatorowi. Skorzystano więc z prostego klasyfikatora binarnego: **SVM**, który wykorzystywany jest do porównywania zbiorów danych o dwóch różnych etykietach. Przy okazji wizualizacji wyznaczana jest też wartość liczbową dokładności takiego klasyfikatora. Użycie tego konkretnego algorytmu pozwoli na podkreślenie ewentualnych problematycznych par podobnych zbiorów danych.

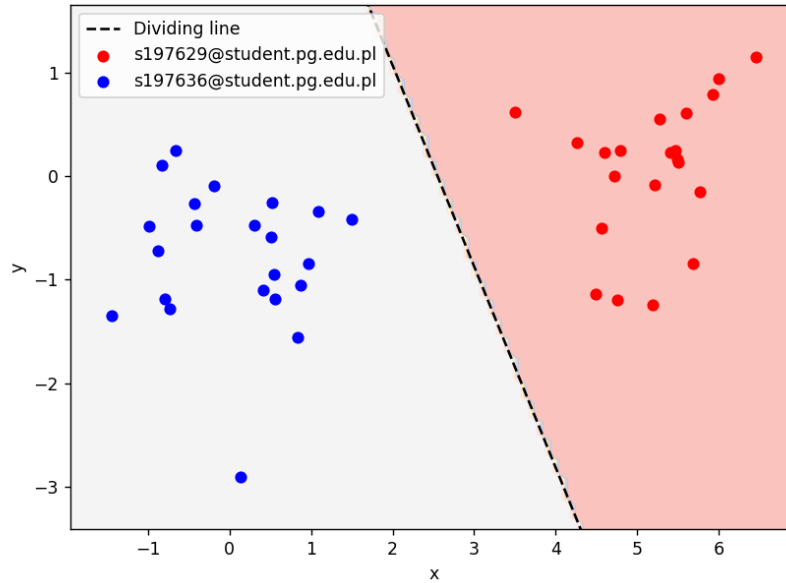
Opis przeprowadzonej ewaluacji

2.3.1 SVM - Support Vector Machine

Za pomocą klasyfikatora SVM, jesteśmy w stanie podzielić zbiór na dwa, poprzez znalezienie odpowiedniej hiperpłaszczyzny maksymalizującej margines między dwoma klasami. W związku z tym, że

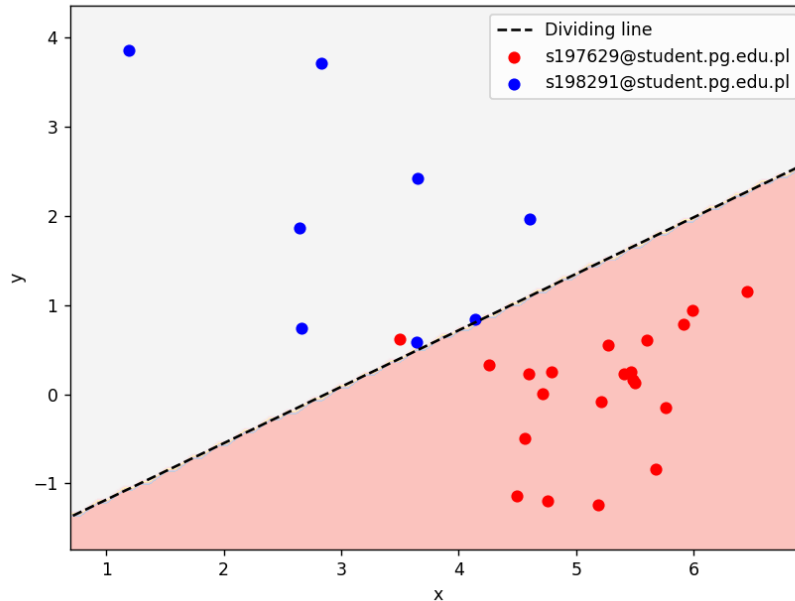
jest on klasyfikatorem binarnym, należało wywołać go osobno dla każdej możliwej pary etykiet w ogólnym zbiorze zredukowanych cech. W ten sposób uzyskujemy podział dla wszystkich kombinacji oraz związaną z nimi dokładność. Poniżej przykłady dla podzbioru 3 osób.

s197629@student.pg.edu.pl and s197636@student.pg.edu.pl divided by linear SVM

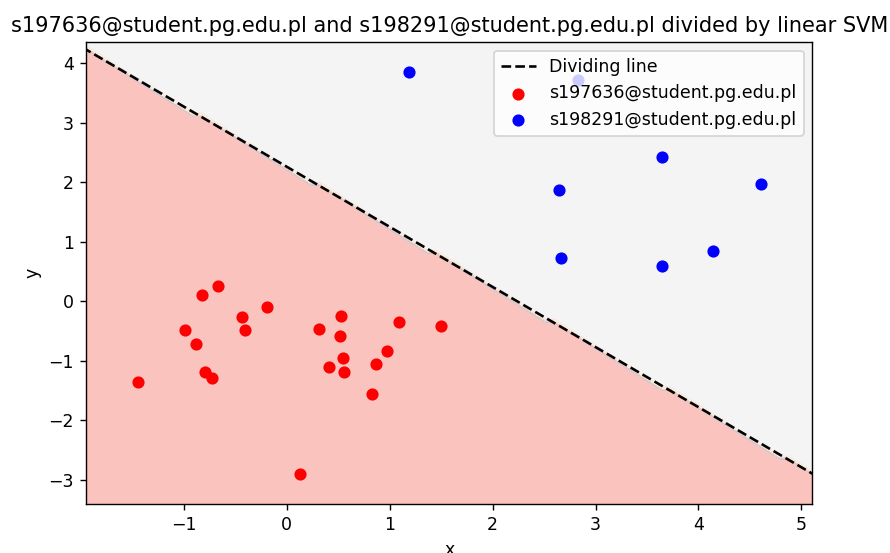


Rysunek 2.7: Wykres przedstawiający podział klasyfikatorem SVM dla dwóch klas
Zmierzona dokładność: Accuracy for s197629 and s197636: 100.00%

s197629@student.pg.edu.pl and s198291@student.pg.edu.pl divided by linear SVM



Rysunek 2.8: Wykres przedstawiający podział klasyfikatorem SVM dla dwóch klas
Zmierzona dokładność: Accuracy for s197629 and s198291: 96.55%



Rysunek 2.9: Wykres przedstawiający podział klasyfikatorem SVM dla dwóch klas
Zmierzona dokładność: Accuracy for s197636 and s198291: 100.00%

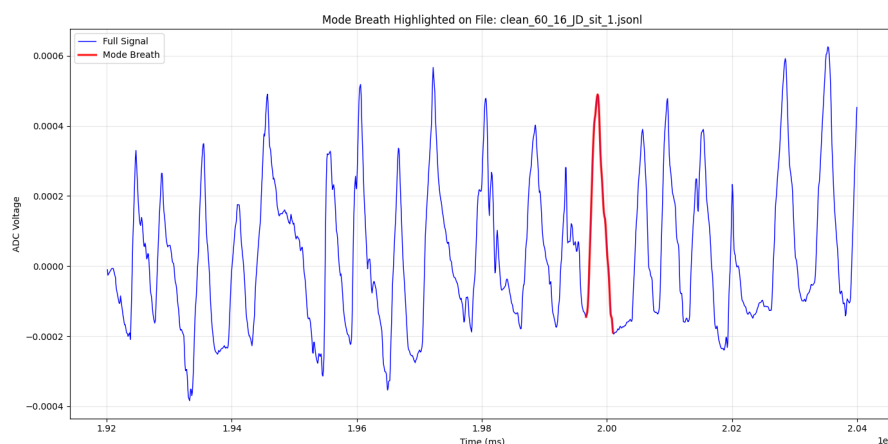
2.4 Tworzenie profili oddechowych osób

Ostatnim etapem analizy danych jest wytworzenie unikatowego i uśrednionego profilu oddechowego dla każdej osoby objętej badaniem na podstawie szeregu wyznaczonych cech. Proces ten pozwala na wyznaczenie wzorca charakterystycznego oddechu dla konkretnego użytkownika, co może służyć jako punkt odniesienia w dalszych badaniach nad biometryczną identyfikacją lub diagnostyką. Wytworzony profil zawiera zarówno uśrednione wartości kluczowych cech czasowo-amplitudowych, jak i reprezentatywny sygnał oddechowy - zestaw 10 oddechów będących dominantą wśród analizowanych danych.

2.4.1 Metodyka wyznaczania profilu

Tworzenie profilu opiera się na następujących operacjach:

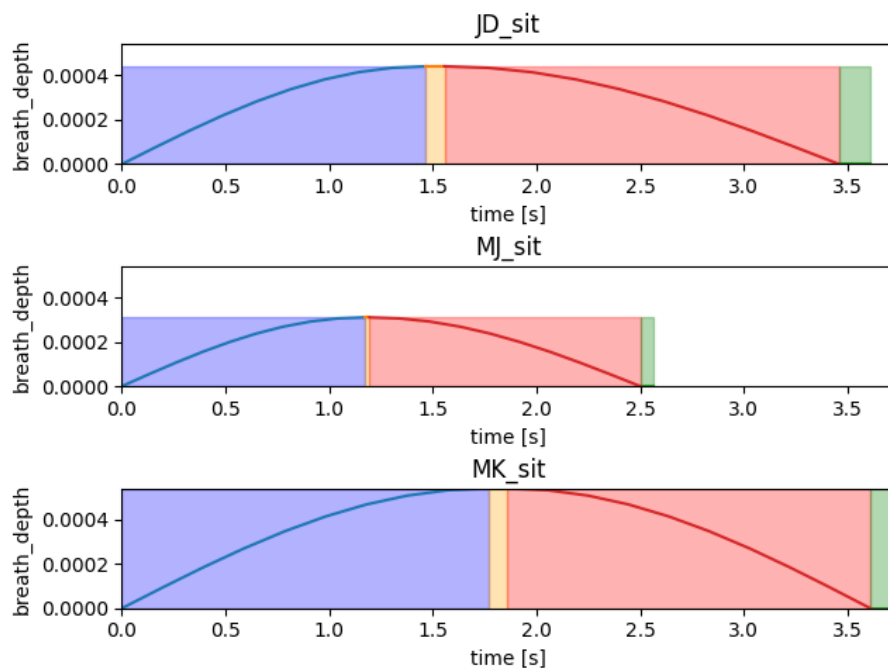
- Wyznaczaniu parametrów liczbowych każdego oddechu w zbiorze danych danej osoby.
- Uśrednianiu oraz wyznaczaniu dominanty tych parametrów.
- Selekcji najbardziej reprezentatywnego cyklu oddechowego na podstawie tych danych obarczonych eksperymentalnie ustalonymi wagami.
- Zapisaniu tych parametrów do dalszej analizy oraz wizualizacji.



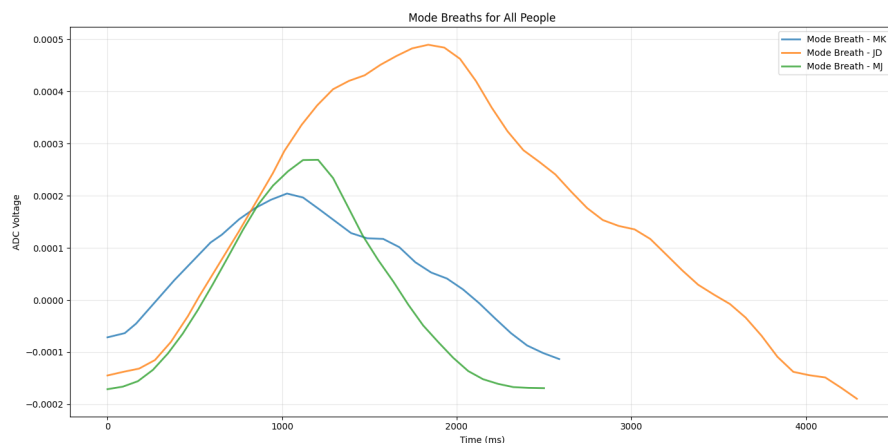
Rysunek 2.10: Przykładowy cykl oddechowy stanowiący najbardziej reprezentatywną próbkę.

Warto zaznaczyć, że przy tworzeniu najbardziej reprezentatywnego oddechu obecnie stosujemy dwa podejścia:

- **Generowanie syntetycznego sygnału:** Na podstawie wyznaczonych parametrów (amplituda, czas trwania, długości faz oddechowych) generowany jest sztuczny sygnał oddechowy przy użyciu funkcji sinusoidalnych i odpowiednich przesunięć fazowych.
- **Wybór rzeczywistego cyklu oddechowego:** Zamiast generować sztuczny, syntetyczny sygnał, algorytm przeszukuje bazę zarejestrowanych i oczyszczonych oddechów danej osoby. Wybierany jest ten cykl, którego parametry (amplituda, czas trwania i długości poszczególnych faz oddechowych) znajdują się najbliżej wyliczonego matematycznie profilu.



Rysunek 2.11: Przykładowa reprezentacja graficzna profilu oddechowego w postaci wygenerowanego syntetycznego sygnału.



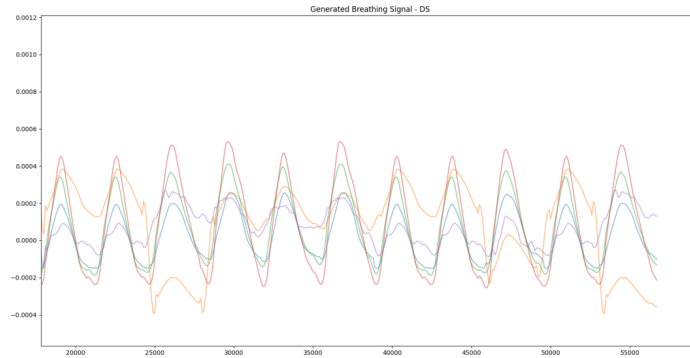
Rysunek 2.12: Przykładowa reprezentacja graficzna profilu oddechowego w postaci wybranego rzeczywistego cyklu oddechowego.

2.4.2 Zastosowanie profilu

Na chwilę obecną wygenerowany profil oddechowy pełni jedną niezwykle istotną funkcję w projekcie:

- **Weryfikacja jakościowa:** Pozwala na szybkie porównanie wizualne „typowego” oddechu różnych osób, co potwierdza zasadność użycia algorytmów takich jak MDS czy PCA do ich separacji.

- **Gererowanie segmentów w celu testowania modelu:** Posiadając zbiór najbardziej reprezentatywnych oddechów dla danej osoby można w prosty sposób wygenerować sygnał oddechowy, który powinien stanowić statystycznie poprawną próbkę oddechu danej osoby. Można np. wygenerować segment (zgodnie z zadanymi kryteriami - na chwilę obecną jest to czas trwania) i poddać do klasyfikacji zarówno przy użyciu zestawu wyznaczonych, interpretowalnych cech, jak i *TSC*.



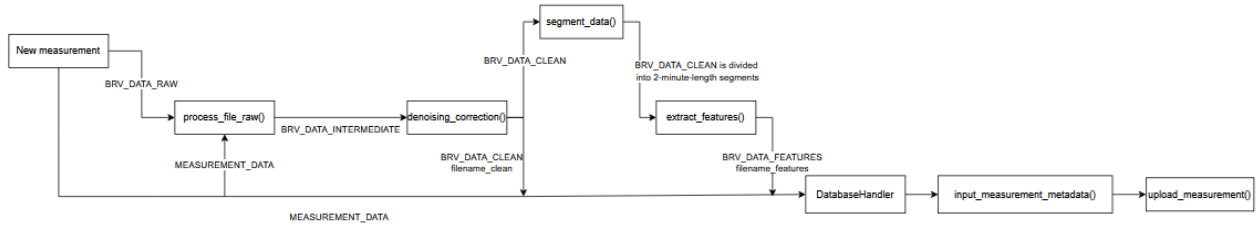
Rysunek 2.13: Przykładowa reprezentacja sygnału oddechowego wygenerowanego w oparciu o powyższy opis.

3 Kod źródłowy - toolkit

Struktura kodu źródłowego została zaprojektowana w sposób modularny, co pozwala na separację logiki przetwarzania sygnałów, ekstrakcji cech oraz wizualizacji rezultatów. Całość implementacji podzielono na trzy główne segmenty funkcjonalne.

3.1 Przepływ sterowania w głównym potoku przetwarzania data_processing_pipeline

Poniżej umieszczona jest wizualizacja przepływu danych przy wywoływaniu skryptu `data_processing_pipeline.py`. Zawiera on wywołania z 3 poniższych skryptów w celu przetworzenia i ekstrakcji cech z naszego sygnału, a na koniec wysyła tak przetworzony pomiar do bazy danych.



Rysunek 3.1: Diagram przedstawiający przepływ danych w potoku `data_processing_pipeline`

3.1.1 Klasy wykorzystywane w potoku przetwarzania danych

- **BRVDataIntermediate**

Tabela 3.1: Opis struktury klasy `BRVDataIntermediate`.

Pole	Typ	Opis
timestamps	NDArray[int64]	stempel czasowy próbki
adc_normalized_data	NDArray[float64]	dane z pasów po znormalizowaniu
signal_maxima	NDArray[float64]	wartości oddechów sygnału objętego przetwarzaniem
signal_minima	NDArray[float64]	wartości dolin sygnału objętego przetwarzaniem

- **BRVDataClean**

Tabela 3.2: Opis struktury klasy `BRVDataIntermediate`.

Pole	Typ	Opis
timestamps	NDArray[int64]	stempel czasowy próbki
adc_data	NDArray[float64]	dane z pasów po oczyszczeniu

- **BRVDataFeatures**

Postać analogiczna do postaci wektora cech.

```
{
  "bpm": NDArray[float64]
  "avg_breath_depth": NDArray[float64]
  "avg_breath_depth_std_dev": NDArray[float64]
  "phases_avg_values": NDArray[float64]
  "breath_shape": NDArray[float64]
  "breath_length_variability": NDArray[float64]
  "breath_amplitude_variability": NDArray[float64]
  "belt_share": NDArray[float64]
  "belt_share_std": NDArray[float64]
}
```

3.2 Architektura systemu

Kod realizuje pełny potok przetwarzania danych (pipeline) – od surowych odczytów z sensorów kamizelki, aż po wizualizację wyników klasyfikacji. Główne moduły to:

- **Przetwarzanie wejścia (Initial processing):** Obsługa plików JSONL, parsowanie ich na format odpowiedni do analizy, czyli wartości z 5 pasów na przestrzeni czasu.
- **Przetwarzanie wejścia (Outlier detection):** czyszczenie danych (anomalie), wygładzanie spline-ami oraz segmentacja na 2-minutowe interwały.
- **Wykrywanie cech (Feature extraction):** Przekształcenie sygnału czasowego na wektory cech (np. BPM, głębokość oddechu, fazy oddechu).

Dodatkowo, system uwzględnia również wizualizację danych jako osobny, opcjonalny etap.

- **Wizualizacja danych:** Implementacja algorytmów redukcji wymiarowości (PCA, MDS) oraz ewaluacja separowalności klas przy użyciu SVM.
- **Utworzenie połączenia oraz obsługa zapytań do API:** Warstwa interakcji z BRICS API w celu przesyłu oraz odbioru wyników przetwarzania pomiaru
- **Interakcja z archiwami pomiarów:** Warstwa do wydobycia pobranych pomiarów w celu ich dalszego przetwarzania

3.3 Kluczowe funkcje i przepływ danych wewnątrz skryptu

`initial_data_processing`

Przepływ danych opiera się na klasie `ADC_data`, która przechowuje obrabiane dane na różnych etapach przetworzenia, w tym znormalizowane wyniki pomiarów w voltach. Poniżej przedstawiono kluczowe funkcje procesowe:

1. `process_file()` oraz `handle_input_data()`: Odpowiadają za parsowanie surowego formatu JSON (zawierającego dane z akcelerometrów IMU i tensometrów ADC) do ustandaryzowanych obiektów procesowych.
2. `breath_separation()`: Wykorzystuje analizę ekstremów lokalnych do podziału ciągłego sygnału na pojedyncze cykle oddechowe.
3. `outlier_detection()`: Realizuje filtrację statystyczną, usuwając $x\%$ oddechów o najbardziej odbiegających parametrach czasowo-amplitudowych, co minimalizuje wpływ błędów pomiarowych.

4. `split_data_into_segments()`: Normalizuje długość próbek poprzez resampling i dzieli sygnał na fragmenty ułatwiające dalszą analizę statystyczną.

3.4 Kluczowe funkcje i przepływ danych wewnątrz skryptu `extract_features`

Skrypt ten odpowiada za wyznaczenie wektorów cech dla segmentów oczyszczonego sygnału stanowiącego wyjście poprzedniego skryptu. Realizuje on to przez wybranie segmentów, których procent utraconych danych nie przekroczył wartości `ACCEPTABLE_DATA_LOSS` ustawionej bazowo na 0.2

- `basic_feature_extraction()`: Funkcja, w której śródku następuje ekstrakcja wektora cech. Jest ona wywoływana pojedynczo, dla każdego segmentu analizowanego przez skrypt (mniej niż 20% utraty danych). Procedury wykonywane w ramach tej funkcji zostały opisane w sekcji 2.2.5.

3.5 Kluczowe funkcje i przepływ danych wewnątrz skryptu `visualize_data`

Skrypt ten odpowiada za wizualizację danych po przetworzeniu i ekstrakcji cech. Kluczowe funkcje i elementy stanowiące część wizualizacji to:

- `NO_OF_FEATURES_AFTER_ALG`: Stała mówiąca o tym, do ilu cech należy zredukować nasze wektory.
- `FeatureData`: Struktura trzymająca dane zarówno przed, jak i po redukcji cech, etykiety dla danych oraz kolory do wizualizacji.
- `feature_loading`: Funkcja zajmująca się wczytaniem wektorów cech z pliku `extracted_features.jsonl` do wyżej wspomnianej struktury.
- `MDS_algorithm`: Funkcja wywołująca algorytm redukcji cech MDS i wyświetlająca jej wynik na ekranie wraz z etykietami danych.
- `PCA_algorithm`: Funkcja wywołująca algorytm redukcji cech PCA i zapisująca wynik w strukturze.
- `SVM_validation`: Funkcja mająca na celu walidację możliwości podziału zbioru binarnie przez hiperpłaszczyznę.
- `plotheatmap`: Funkcja standaryzująca dane do przedziału [0,1] (dla każdej cechy osobno) i wyświetlająca je za pomocą heatmapy.

3.6 Kluczowe funkcje i przepływ danych wewnątrz skryptu `create_data_profiles`

Skrypt ten odpowiada za tworzenie indywidualnych profili oddechowych każdej osobie, dla której zostały zebrane dane. Dzięki temu możliwe jest przedstawienie i dostrzeżenie indywidualnych cech oddechowych każdej osoby i wizualne potwierdzenie zasadności przeprowadzanego badania. Każdy taki profil jest Pythonową mapą i zawiera:

- `signal`: Jest to wycinek surowego sygnału oddechowego danej osoby zawierający jeden oddech, który statystycznie najlepiej reprezentuje jej sposób oddychania ze względu na szereg parametrów stanowiących kolejne pola tej mapy.
- `timestamps`: Odpowiada za przechowywanie znaczników czasowych odpowiadających kolejnym punktom sygnału.

- `inhale`, `inspiratory_phase`, `exhale`, `expiratory_phase`: Cztery pola odpowiadające kolejnym fazom oddechu. Każde z nich zawiera różnicę w timestampach pomiędzy początkiem i końcem danej fazy.
- `length`: Długość całego oddechu w milisekundach.
- `depth`: Głębokość oddechu, czyli różnica pomiędzy maksymalną a minimalną wartością sygnału w danym oddechu.

Do zadań na przyszłość należy rozszerzenie tej mapy o dodatkowe pola zawierające cechy, które dokładniej pozwolą charakteryzować indywidualny sposób oddychania każdej osoby. Poniżej znajdują się najważniejsze funkcje odpowiadające za tworzenie profili oddechowych:

- `create_data_profiles()`: Główna funkcja tworząca profile oddechowe dla każdej osoby na podstawie przetworzonych danych.
- `get_people_list()`: Funkcja zwracająca listę osób na podstawie etykiet danych znajdujących się w folderze `results`.
- `calculate_breath_characteristics()`: Oblicza fazy oddechu (wdech, faza inspiracji, wydech, faza ekspiracji) na podstawie sygnału dla każdego wykrytego oddechu.
- `get_mode_breath()`: Wybiera najczęściej występujący oddech na podstawie cech statystycznych.
- `visualize_profiles()`: Tworzy wykresy przedstawiające profile oddechowe każdej osoby.

3.7 Utworzenie połączenia oraz obsługa zapytań do API - DatabaseHandler

DatabaseHandler to klasa odpowiadająca za utworzenie środowiska oraz obsługę przygotowanych scenariuszy zapytań do BRICS API. Udostępnia one następujące metody:

- `upload_measurement()`: Wykonuje wysłanie pomiaru oraz jego metadanych do API, co w rezultacie powoduje jego zapis w bazie danych.
- `download_measurement()`: Daje możliwość pobrania pomiarów odpowiadających parametrom wyszukiwania.
- `download_measurement()`: Odpowiada za przesłanie żądania usunięcia pomiaru o danym identyfikatorze.

3.8 Interakcja z archiwami pomiarów

MeasurementDatasetHook to klasa odpowiadająca za automatyczne wydobywanie oraz udostępnienie pomiarów znajdujących się w pobranych archiwum ZIP. Dzięki implementacji wewnętrznego iteratora, możliwe jest następujące użycie klasy:

```
mdh = MeasurementDatasetHook(target: raw)

for file in mdh:
    # procesowanie pliku z surowymi danymi
```

3.9 BRICStoolkit

Ze względu na wysoki stopień ponownego wykorzystywania kodu wykonanego w ramach repozytorium `toolkit` w innych częściach projektu, szczególnie w implementacji modelu - większość rozwiązań opisanych w tej sekcji została zebrana w jedną bibliotekę Python - `brics_toolkit`, możliwą do zainstalowania w lokalnym środowisku deweloperskim. Jej instalację można przeprowadzić w następujący sposób używając menedżera bibliotek PIP oraz środowiska venv:

```
git clone https://github.com/BRICScore/toolkit.git
pip install -e toolkit
```

W ten sposób rozwiązano problem konieczności kopiowania kodu w trakcie pracy członków zespołu.

4 Serwer BRICS

W celu usprawnionego przechowywania informacji zebranych w trakcie realizacji projektu grupowego utworzono serwer BRICS, udostępniony dla członków zespołu. Rozdział ten ma na celu omówić zaimplementowane zabezpieczenia oraz usługi realizowane przez serwer. Obecnie Serwer BRICS jest realizowany w postaci maszyny używającej narzędzia Docker w celu uruchomienia 5 rodzajów kontenerów:

- **BRICS DB** - bazę danych MongoDB
- **BRICS API** - REST API stanowiące interfejs do BRICS DB
- **Redis** - baza danych stanowiąca pośrednika dla kolejki zadań
- **Celery Worker** - kontener z procesem realizującym zadania z kolejki
- **Celery Beat** - kontener odpowiadający za regularne dodawanie zadań do kolejki

Kolejne sekcje będą miały na celu szczegółowe omówienie realizacji poszczególnych usług powyżej.

4.1 BRICS DB

W celu przechowywania danych użyto systemu MongoDB. Jest to nierelacyjna baza danych, przechowująca informacje w postaci kolekcji zawierających dokumenty.

Baza danych zawiera obecnie jedną kolekcję (poza domyślnymi utworzonymi przez MongoDB):

- **measurements** - dokumenty zawierające informacje na temat pomiarów przechowywanych w systemie plików serwera

Tabela 4.1: Opis struktury obiektu MeasurementMetadata

Pole	Typ	Opis
_id	str	identyfikator pomiaru
timestamp	float	timestamp odpowiadający dacie rozpoczęcia pomiaru
duration_ms	int	długość pomiaru w milisekundach
filepath_raw	str	ścieżka do pliku zawierającego surowe dane pomiarowe
filepath_clean	str	ścieżka do pliku zawierającego oczyszczone dane
filepath_features	str	ścieżka do pliku zawierającego cechy
labels	LabelsData	Obiekt przechowujący dane dot. etykiet i osoby badanej

Tabela 4.2: Opis struktury klasy LabelsData

activity	str	typ aktywności spośród wybranych
person_data	BioData	Obiekt przechowujący dane dot. osoby badanej

Tabela 4.3: Opis struktury klasy BioData

person_id	str	identyfikator osoby badanej
age	int	wiek osoby badanej
gender	str	biologiczna płeć osoby badanej
health	str	opis stanu zdrowia
condition	str	kondycja fizyczna osoby badanej spośród wybranych
weight	int	waga osoby badanej w kilogramach
height	int	wzrost osoby badanej w centymetrach


```

_id: ObjectId('69dfa0449406c7beccacabe8'),
duration_ms: 1029207,
filepath_clean: '/dataset/69dfa0449406c7beccacabe8_clean',
filepath_features: '/dataset/69dfa0449406c7beccacabe8_features',
filepath_raw: '/dataset/69dfa0449406c7beccacabe8_raw',
labels: {
  activity: 'sit',
  person_data: {
    person_id: 's198291@student.pg.edu.pl',
    age: 22,
    gender: 'male',
    health: 'healthy',
    condition: 'regular',
    weight: 69,
    height: 172
  }
},
timestamp: 1776263170.8026357

```

Rysunek 4.1: Przykładowy rekord metadanych pomiaru

- **models** - najlepsza wersja modelu używana w systemie BRICS

Tabela 4.4: Opis struktury klasy ModelMetadata

Pole	Typ	Opis
<code>_id</code>	str	identyfikator modelu
<code>timestamp</code>	float	timestamp odpowiadający dacie przesłania modelu
<code>filepath_weights</code>	str	ścieżka do pliku zawierającego wagi i hiperparametry modelu
<code>filepath_pth</code>	str	ścieżka do pliku zawierającego obraz modelu w postaci pliku <i>.pth</i>
<code>filepath_scaler</code>	str	ścieżka do pliku zawierającego obiekt StandardScaler przygotowany na danych przetworzonych podczas trenowania modelu

Ze względu na konieczność ochrony wytworzonego modelu, publicznie udostępniane są jedyne algorytmy oraz narzędzia potrzebne do jego wytworzenia - sama implementacja modelu jest przechowywana jedynie w bazie danych lub lokalnie, po pobraniu i/lub wytworzeniu takiego, u członka zespołu projektowego.

4.2 API

W celu komunikacji z wyżej wymienioną bazą danych zostało stworzone REST API, mające na celu ograniczenie dostępu oraz utworzenie warstwy abstrakcji do interakcji z bazą danych. API zostało zrealizowane z użyciem framework'a FastAPI oraz implementacji serwera WWW w technologii Uvicorn.

4.2.1 Punkty końcowe API

By skorzystać z API udostępnione zostały endpoint'y:

- **GET /measurements/download** - służący do pobierania pomiarów odpowiadających parametrom wyszukiwania
- **PUT /measurements/upload** - służący do umieszczania oraz edycji wykonanych pomiarów
- **DELETE /measurements/delete** - służący do usuwania pomiarów

Analogicznie, dla modelu istnieją:

- **GET /models/download** - służący do pomiaru najnowszej wersji modelu

- **PUT /measurements/upload** - służący do aktualizacji najlepszej wersji modelu
- **DELETE /measurements/delete** - służący do usunięcia modelu

Dostęp do nich jest realizowany w ramach programu poprzez aplikację stanowiącą warstwę interakcji z kamizelką oraz skrypty dostępu, stanowiące realizację warstwy użytkownika tworzącego zbiór danych, pozwalające na wybranie pliku oraz dodanie odpowiednich metadanych, bądź podanie parametrów wyszukiwania.

4.2.2 Szczegółowa realizacja punktów końcowych

Sekcja ta ma na celu dać wgląd w działanie każdego z punktów końcowych.

GET /measurements/download

W żądaniu HTTP do tego punktu końcowego przekazywany jest ciąg znaków będący kwerendą zawierającą filtry wyszukiwania pomiaru. Przetwarzanie tego zapytania można sprowadzić do następujących faz:

1. Utworzenie poprawnej kwerendy do MongoDB na podstawie parametrów żądania
2. Wyszukiwanie metadanych na podstawie kwerendy
3. Wydobywanie plików pomiarowych i stworzenie archiwum zip zawierającego pomiary odpowiadające parametrom wyszukiwania
4. Wysłanie odpowiedzi na żądanie
5. Usunięcie folderów tymczasowych utworzonych w ramach przetwarzania żądania

PUT /measurements/upload

W żądaniu HTTP do tego punktu końcowego są przekazywane archiwum zip zawierające trzy pliki będące trzema wersjami oryginalnego pliku pomiarowego oraz ciąg znaków w formacie JSON zawierające metadane tego pomiaru. Przetwarzanie tego zapytania składa się z następujących faz:

1. Weryfikacja poprawności formatu metadanych
2. Wyznaczenie, czy następuje aktualizacja pomiaru czy utworzenie nowego - poprzez wyszukiwanie dostarczonego identyfikatora pomiaru w bazie danych
3. Stworzenie struktur potrzebnych do przekazania procesu finalizacji zapisu pomiaru do kolejki zadań
4. Utworzenie zadania finalizacji zapisu pomiaru do bazy danych oraz systemu plików
5. Odesłanie odpowiedzi o przyjęciu lub odrzuceniu pliku do przetwarzania

Zadanie finalizacji zapisu zostało omówione w sekcji zadań.

DELETE /measurements/delete

W żądaniu HTTP do tego punktu końcowego przekazywany jest jedynie identyfikator pomiaru do usunięcia z bazy danych. Przetwarzanie składa się z następujących faz:

1. Wyszukiwanie metadanych zawierających podany identyfikator
2. Usunięcie metadanych oraz plików pomiarowych w przypadku znalezienia dokumentu

GET /models/download

W żądaniu HTTP do tego punktu końcowego, ze względu na jego prostotę, nie jest przekazywany żaden parametr lub obiekt. Przetwarzanie tego zapytania można sprowadzić do następujących faz:

1. Wyszukiwanie metadanych modelu w bazie danych
2. Wydobycie plików modelu i stworzenie archiwum zip z nich złożonych
3. Wysłanie odpowiedzi na żądanie
4. Usunięcie folderów tymczasowych utworzonych w ramach przetwarzania żądania

PUT /models/upload

W żądaniu HTTP do tego punktu końcowego, podobnie jak w przypadku plików pomiarowych, część żądania jest procesowana przez proces w tle. Przetwarzanie tego zapytania można sprowadzić do następujących faz:

1. Weryfikacja poprawności formatu metadanych
2. Wyznaczenie, czy następuje aktualizacja modelu czy utworzenie nowego
3. Stworzenie struktur potrzebnych do przekazania procesu finalizacji zapisu modelu do kolejki zadań
4. Utworzenie zadania finalizacji zapisu modelu do bazy danych oraz systemu plików
5. Odesłanie odpowiedzi o przyjęciu lub odrzuceniu pliku do przetwarzania

DELETE /models/delete

Finalnie, w tym żądaniu HTTP, podobnie jak w przypadku pomiarów, następuje proste usunięcie rekordu modelu i jego plików. Przetwarzanie tego zapytania można sprowadzić do następujących faz:

1. Wyszukiwanie metadanych zawierających podany identyfikator
2. Usunięcie metadanych oraz plików modelu w przypadku znalezienia dokumentu

4.2.3 Redis

W celu zaimplementowania kolejki zadań użyto bazy danych Redis, znajdującej się całkowicie w pamięci RAM komputera. Ze względu na istnienie do 16 oddzielnych baz danych w ramach jednej instancji Redis (ponumerowanych od 0 do 15), jako kolejki zadań użyto bazy danych nr 0, a jako rejestru stanów wykonywanych oraz wykonanych zadań użyto bazy danych nr 1.

Należy zaznaczyć, że wybór ten jest czysto arbitralny. Zdecydowano się na przechowywanie stanu zadań wykonanych ze względu na rozszerzalność projektu w przyszłości (np. poprzez usługę typu dashboard, w której to zespół tworzący zbiór danych mógłby łatwo sprawdzać stan systemu).

4.2.4 Celery Worker

Technologia Celery pozwala w ramach projektu na utworzenie kontenerów realizujących kolejkę zadań z bazy danych Redis. Pracownik taki przyjmuje zadanie z kolejki, po czym je wykonuje, a jego rezultat zwraca z powrotem do bazy danych. Dzięki zastosowaniu tej technologii wyznaczono następujące zadania:

- Finalizacja procesu przesłania lub aktualizacji pomiaru/modelu
- Codzienny automatyczny zrzut bazy danych oraz plików pomiarów do kopii zapasowej

Finalizacja procesu przesłania lub aktualizacji pomiaru/modelu

Proces ten można rozbić na następujące podzadania:

1. Weryfikacja przekazanych metadanych oraz plików pomiarowych
2. Ekstrakcja i konwersja formatu plików pomiarowych/modelu
3. Atomiczna insercja/aktualizacja plików pomiarowych/modelu w systemie plików
4. Atomiczna insercja/aktualizacja rekordu metadanych w bazie danych

Dzięki realizacji części tego procesu w postaci zadania w kolejce czas wysyłania pomiaru jest znacznie skrócony, co jest głównym sposobem interakcji z Serwerem BRICS.

Codzienny automatyczny zrzut bazy danych oraz plików pomiarów do kopii zapasowej

Proces ten można rozbić na następujące podzadania:

1. Utworzenie kopii zapasowej bazy danych MongoDB
2. Utworzenie kopii zapasowej plików pomiarowych oraz modeli

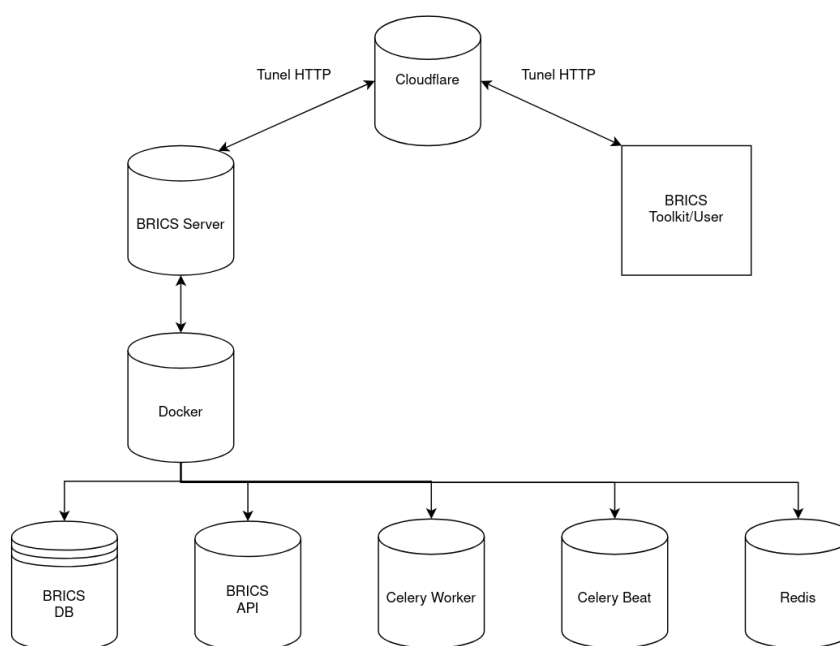
Dzięki realizacji części tego procesu w postaci zadania w kolejce czas wysyłania pomiaru jest znacznie skrócony, co jest głównym sposobem interakcji z Serwerem BRICS.

4.2.5 Celery Beat

W ramach technologii Celery istnieje również rozwiązanie Celery Beat, pozwalające na regularne dodawanie zadań do kolejki wedle określonych zasad. W ten sposób zrealizowane jest codzienne wykonywanie kopii zapasowych plików na serwerze.

4.3 Zabezpieczenia

Sekcja ta ma na celu wyszczegółowanie rozwiązań zastosowanych w celu realizacji bezpiecznego dostępu członków projektu do serwera BRICS oraz usług przez niego realizowanych. Ich implementację można przedstawić w postaci poniższego diagramu.



Rysunek 4.2: Uproszczony diagram transmisji danych wewnątrz projektu (pomijając kamizelkę BRV)

4.3.1 Konsola zdalna

Serwer BRICS udostępnia członkom zespołu usługę konsoli zdalnej poprzez SSH. Dla każdego z nich utworzono parę kluczy, i jedynym możliwym sposobem dostępu do konsoli jest uwierzytelnienie poprzez takową parę kluczy, efektywnie tworząc whitelistę, zawierającą jedynie określonych członków zespołu. Usługa SSH dostępna jest jedynie z sieci lokalnej serwera, tym samym ustanawiając tunel Cloudflare jako jedyne zdalne medium połączenia.

Przykładowy sposób ustanowienia konsoli zdalnej na serwerze BRICS z maszyny członka zespołu, dla użytkownika brics (główny użytkownik):

```
sudo cloudflared access ssh --hostname brics-ssh.electimore.xyz --url localhost:5555  
ssh brics@localhost -p 5555
```

4.3.2 API

Dostęp do API możliwy jest jedynie poprzez okazanie odpowiedniej wartości (Service Token) w nagłówkach żądań HTTP. Wartość ta jest zmienną środowiskową, przekazaną każdemu z członków zespołu. Żądania jej niezawierające zostaną odrzucone już na poziomie Cloudflare'a.

4.3.3 Docker

Do reszty usług bezpośredni dostęp jest niemożliwy ze względu na ustawienia zapory sieciowej serwera. Należy się wpierw połączyć poprzez konsolę zdalną, by móc działać bezpośrednio z resztą skonteneryzowanych usług.

5 Płytką pośrednicząca w komunikacji

Aby usprawnić pobieranie danych przy pomocy kamizelki BRV utworzona została płytką pośrednia, która ma za zadanie odbierać dane przekazywane szeregowo i wysyłać je dalej przy pomocy technologii Bluetooth. W tym rozdziale omówiona zostanie budowa płytki, oraz struktura zaimplementowanego oprogramowania. W tym momencie płytką pośrednia udostępnia następujące usługi:

- **BRICS BLE** - serwer Bluetooth LE w trybie Peripheral
- **BRICS Serial** - wielowątkowa aplikacja pobierająca szeregowo dane z kamizelki BRV
- **BRICS.service**

Usługi te są dostępne razem z podłączoną kamizelką BRV i mają na celu jedynie usprawnienie przekazywania danych i ograniczenie utraty danych podczas pomiarów.

5.1 Architektura płytki

Płytką została zrealizowana przy pomocy mikroprocesora Raspberry Pi Zero W, na którym zainstalowany został system operacyjny Raspbian 32-bit. Posiada 64 GB pamięć w postaci karty micro SD.

5.2 Wielowątkowa aplikacja pośrednicząca

Do obsługi współbieżnego przetwarzania danych zaimplementowana została wielowątkowa aplikacja. Uruchomienie aplikacji powoduje uruchomienie trzech wątków:

- **BRVBluetoothServer** - wątek obsługujący działanie serwera Bluetooth, którego specyfikacja zostanie omówiona w dalszej części dokumentu.
- **BRVSerialConnection** - wątek obsługujący szeregowe pobieranie pakietów danych z kamizelki BRV.
- **BRVLedDisplay** - wątek obsługujący sygnalizowanie przy pomocy dołączonych diod o aktywności i stanie pracy kolejnych komponentów.

Komunikacja pomiędzy wątkami została zrealizowana przy pomocy bibliotek:

- **queue** - biblioteka udostępniająca kolejkę FIFO z wbudowanym mechanizmem blokowania
- **asyncio** - biblioteka umożliwiająca asynchroniczne uruchamianie kodu
- **threading** - biblioteka umożliwiająca tworzenie i kontrolowanie wątków

Dane otrzymane na porcie szeregowym są umieszczane w tej kolejce i zdejmowane przez wątek obsługujący serwer Bluetooth. W przypadku, w którym aplikacja napotka błąd lub nieobsłużony wyjątek wątki są niszczone i aplikacja jest uruchamiana ponownie.

5.3 Serwer Bluetooth

W celu utworzenia serwera BLE wykorzystane zostały biblioteki:

- **bluez** - podstawowa biblioteka służąca do tworzenia połączeń wykorzystujących technologię Bluetooth
- **bluez-peripheral** - biblioteka służąca do tworzenia serwerów BLE wykorzystujących bibliotekę Bluez, oraz GATT API
- **dbus** - biblioteka służąca do komunikacji z wykorzystaniem głównej szyny danych

Utworzony serwer składa się z dwóch klas, jedna z nich stanowi deklarację serwisu udostępnianego przez serwer Bluetooth, a druga obsługuje wątek uruchamiającego serwer Bluetooth.

Serwer działa w 2 następujących po sobie fazach:

1. **Uruchamianie** - podczas tej fazy serwer musi zadeklarować swoje serwisy, charakterystyki, adapter, agent i rozgłaszanie. Dane o serwisie i charakterystyce dostępne na płycie definiowane są w klasie **BRVDataService**, a sposób rozgłaszania, agent oraz adapter definiowane są podczas inicjalizacji serwera w klasie **BRVBluetoothServer**. Ponadto, podczas inicjalizacji wątek dostaje od systemu szynę danych d-bus, na której wykonywana będzie konieczna komunikacja ze sterownikami Bluetooth i kolejno inicjalizowane są kolejne elementy serwera.
2. **Działanie** - podczas tej fazy kolejno zdejmowane są kolejne pakiety z kolejki FIFO, oraz rozdzielane na mniejsze aby umożliwić ich transport przy pomocy Bluetooth. Aby wysłać pakiet na dostępnej charakterystyce serwisu uruchamiana jest metoda **update_BRV_value**, która powoduje wygenerowanie powiadomienia. Wszystkie urządzenia, które są podłączone pod system powiadomień tej charakterystyki otrzymują informacje o jej zmianie i mogą odczytać wartość

5.3.1 Działanie serwera Bluetooth LE

Serwer Bluetooth LE (Low Energy) składa się z kilku znaczących komponentów. Głównym z nich jest serwis, który zgodnie z protokołem GATT serwis zawiera w sobie charakterystykę. Jest ona ustawiona w tryb Indicate, co umożliwia otrzymywanie komunikatów ACK po wysłaniu pakietów.

5.4 Sygnaizowanie stanu płytki GPIO

W ramach aplikacji sygnalizującej stan pracy płytki obsługiwane jest działanie 3 diod: sygnalizującej pracę płytki, sygnalizującej odbieranie danych przez port szeregowy, oraz sygnalizującej działanie serwera Bluetooth. Sterowanie działaniem diod odbywa się przy pomocy map bitowych, których stan jest regularnie brany pod uwagę podczas działania programu.

Możliwe stany diod:

- dioda sygnalizująca działanie płytki świeci się stałym światłem - płytka działa
- dioda sygnalizująca działanie płytki mruga - działa aplikacja obsługująca przekazywanie danych
- dioda sygnalizująca działanie serwera mruga - działa serwer Bluetooth i dane są aktywnie przekazywane
- dioda sygnalizująca działanie portu szeregowego mruga - pobierane są dane z portu szeregowego

5.5 Przykładowy scenariusz użycia

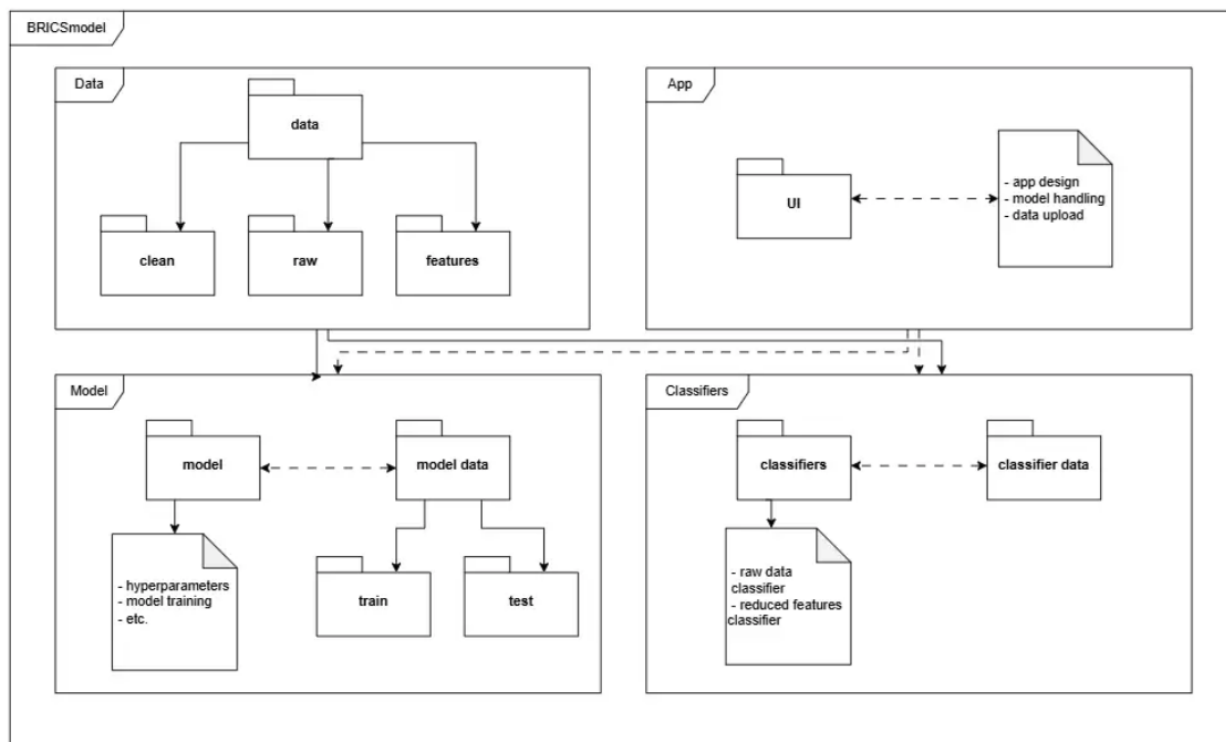
Płytką może zostać użyta w następujący sposób:

1. Podłączenie płytki do zasilania
2. Uruchomienie programu odbierającego dane
3. Zapaliła się dioda sygnalizująca działanie serwera Bluetooth i w programie odbierającym dane widać podłączenie do serwera
4. Podłączenie płytki do kamizelki BRV i rozpoczęcie pomiaru
5. Zakończenie pomiaru i odpięcie kamizelki BRV od zasilania
6. Odłączenie płytki od prądu

6 Model BRICS

6.1 Organizacja repozytorium

Nasze repozytorium modelu podzieliśmy na 4 różne składowe zgodnie z poniższym diagramem pakietów UML:



Rysunek 6.1: Diagram przedstawiający strukturę pakietów w repozytorium BRICSModel

- **Classifiers** - sekcja poświęcona jest klasyfikatorom i ich danym, które funkcjonują jako dodatkowe rozszerzenia naszej aplikacji. Są to głównie skrypty pozwalające na dodatkową wizualizację i przegląd naszych danych poprzez wykonywanie klasyfikacji na różnych stopniach przetwarzania i jakości danych.
- **Data** - jest to głównie przestrzeń przechowywania danych do nauki naszego głównego klasyfikatora oraz dla klasyfikacji wykorzystywanej w aplikacji, działającej na świeżych plikach z kamizelki.
- **Model** - obszar, w którym przechowujemy większość klas i skryptów potrzebnych do wytrenowania i wdrożenia oraz integracji naszego modelu z aplikacją. Przechowywane są tutaj takie pliki jak: wagi modelu, parametry uczenia modelu, skrypty treningowe, czy definicje klas, będących kontenerami na dane.
- **App** - główne miejsce zarządzania pozostałymi elementami repozytorium, w którym integrujemy mechanizmy klasyfikacji, model oraz zarządzanie nim. Jest to zarówno interfejs do posługiwania się repozytorium, jak i nasz końcowy produkt, który mamy na celu rozwijać w projekcie.

6.2 Klasyfikatory

W ramach tej części projektu zaimplementowane zostały dwa klasyfikatory:

- **Klasyfikator danych oczyszczonych** - klasyfikator szeregów czasowych, którymi są dane pozyskane z kamizelki pomiarowej BRV, poddane opisanemu w sekcji 2.2.1 preprocessingowi.
- **Klasyfikator wybranych cech** - klasyfikator umieszczający segmenty pomiarów w przestrzeni dwuwymiarowej, gdzie osie OX i OY stanowią wybrane dwie cechy.

Znajdują się wewnątrz struktury projektu w folderze `classifiers`, gdzie mają dostęp do dedykowanego katalogu na dane pomocnicze o nazwie `classifier_data`.

6.2.1 Klasyfikator danych oczyszczonych

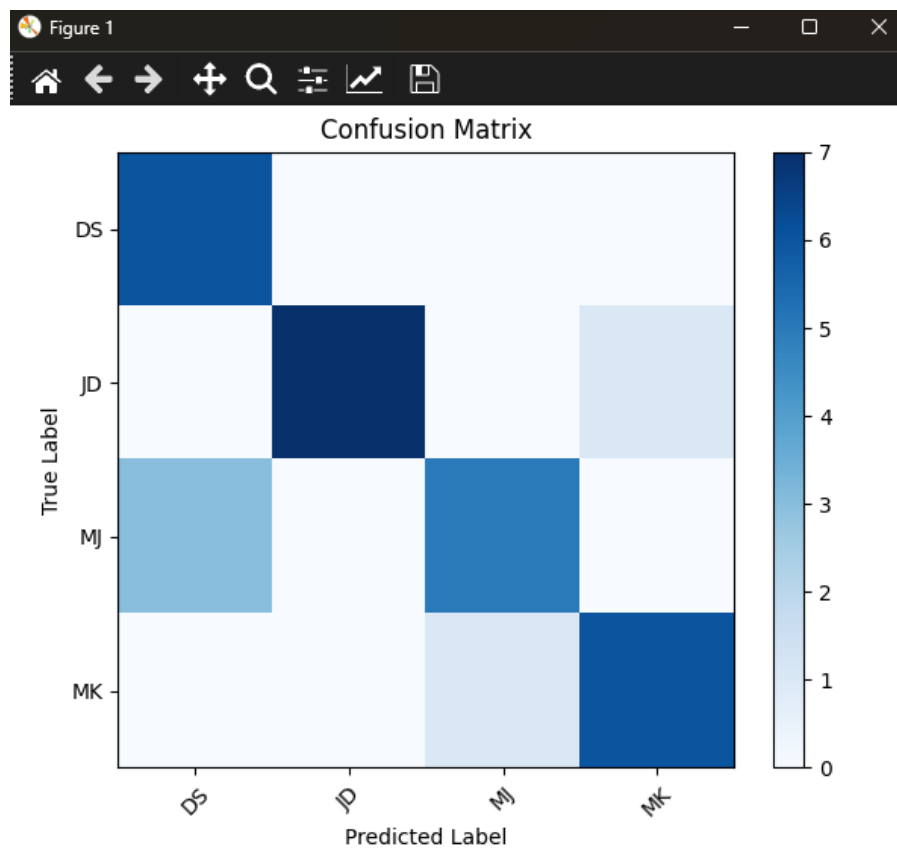
Klasyfikator, jak wspomniano powyżej, pracuje na danych poddanych preprocessingowi, a więc na wejście przyjmuje pliki typu *JSONL* w formacie *clean*. Następnie z nazwy plików pozyskuje etykiety danych oraz przygotowuje dataloadery - treningowy, zawierający 80% zbioru danych oraz testowy, zawierający pozostałe 20%. Zestawy danych zostały wyznaczone przy użyciu funkcji `train_test_split` z biblioteki *sklearn*. W ramach projektu rozważane były również dataloadery wyznaczone za pomocą klasy `GroupShuffleSplit` z tej samej biblioteki, jednakże okazały się mniej efektywne. Ze względu na specyfikę i niewielką liczbę danych wejściowych takie dataloadery prowadziły do grupowania się danych i powstawania znacznych rozbieżności między zbiorami treningowym i testowym - niektóre z klas znajdujących się wewnątrz zbioru testowego nie pojawiały się nigdy w zbiorach treningowych prowadząc do niskiej dokładności na poziomie około **45%**.

Pracujemy na klasyfikatorze LSTM (ang. *Long Short-Term Memory*) z biblioteki *PyTorch* o następujących parametrach:

- `input_size` 5
- `hidden_size` 128
- `num_layers` 2
- `batch_first` True
- `dropout` 0,2
- `bidirectional` True

Powyższe parametry zostały ustalone drogą eksperymentalną i dają dokładność klasyfikacji na poziomie około **80%**.

Wykonany został eksperyment na 4 klasach mający na celu sprawdzić skuteczność opisywanego procesu. Jego wyniki zaprezentowane oraz opisane są poniżej.



Rysunek 6.2: Wykres przedstawiający macierz konfuzji klasyfikatora danych oczyszczonych dla 4 klas

Klasyfikator dla tego scenariusza uzyskał skuteczność na poziomie **82,7%** co stanowiło jeden z najlepszych rezultatów jaki udało się uzyskać. Szczegółowe rezultaty klasyfikacji zostały zawarte w poniższej tabeli:

Tabela 6.1: Reprezentacja wyników zwracanych przez klasyfikator danych oczyszczonych.

	Precyzja	Czułość	Miara-f1	Support
DS	0,6667	1,0000	0,8000	6
JD	1,0000	0,8750	0,9333	8
MJ	0,8333	0,6250	0,7143	8
MK	0,8571	0,8571	0,8571	7
Dokładność	—	0,8276	—	29
Średnia makro	0,8393	0,8393	0,8262	29
Średnia ważona	0,8506	0,8276	0,8269	29

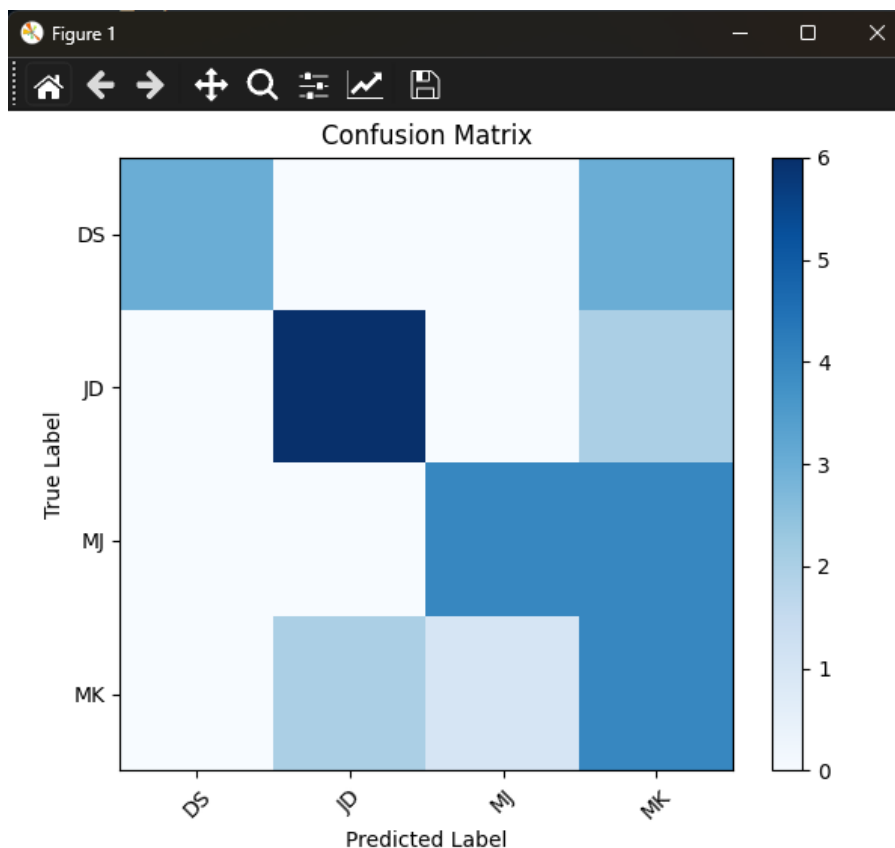
, gdzie wartości w wierszu nagłówkowym oznaczają odpowiednio:

- **Precyzja** - wartość określająca, jaki procent wszystkich przewidywań pozytywnych był rzeczywiście poprawny.
- **Czułość** - wartość określająca, procent wszystkich rzeczywistych przypadków pozytywnych model poprawnie wykrył.
- **Miara-f1** - wartość ta równoważy powyższe dwie stanowiąc ich średnią harmoniczną. Znajduje zastosowanie przy niebalansowanych zbiorach danych
- **Support** - liczba punktów danych danej klasy w zbiorze testowym

Z powyższych wyników można wywnioskować, że model najskuteczniej identyfikował osoby oznaczone etykietami *JD* i *MK* oraz, że mniejsza liczba elementów o etykietach *MJ* i *DS* mogła się przełożyć

na słabsze wyniki w tym obszarze. Dodatkowo widzimy, że mimo wszystko mamy do czynienia ze zbalansowanym zbiorem danych, ponieważ miara-f1 nie odstaje od pozostałych wartości.

Uzyskane rezultaty pokazują zasadność prób identyfikacji osoby (w tym przypadku jest to klasyfikacja do klasy) przy użyciu wyłącznie sygnału oddechowego. Pokazują też w jakim stopniu następuje poprawa względem klasyfikacji po danych czysto surowych - bez przejścia przez preprocessing. Dodatkowo, co zostanie przedstawione w dalszej części dokumentu, zasadnym okazuje się identyfikacja przy użyciu modelu neuronowego - dokładność klasyfikacji wzrasta do ponad 90%.



Rysunek 6.3: Wykres przedstawiający macierz konfuzji klasyfikatora danych surowych dla 4 klas

Powyższy wykres prezentuje wynik klasyfikacji danych surowych z kamizelki pomiarowej, bez preprocessingu. Takie dane osiągają skuteczność klasyfikacji na poziomie około **60%**.

6.2.2 Klasyfikator wybranych cech

Skrypt realizujący tą sekcję umieszcza punkty danych (segmenty) w przestrzeni dwóch cech, które zostały wskazane przez użytkownika albo zostały (jest to opcja domyślna) wybrane automatycznie jako dwie najbardziej skuteczne cechy. Zostało to wyznaczone przy użyciu macierzy kowariancji - funkcja `cov` z biblioteki *NumPy*. Dla każdej pary cech w takiej macierzy wyznaczana jest wartość niesionej informacji, która następnie jest łączona z etykietami tej pary cech i zwracana jako `tuple` tych wartości.

Taka macierz dla naszego zestawu cech prezentuje się następująco i pozwala algorytmowi wyciągnąć następujące wnioski - najlepsza para cech zależy od zbioru danych aktualnie klasyfikowanych, ale wśród najlepszych cech pojawia się zazwyczaj **długość wdechu** oraz **odchylenie standardowe średniej głębokości oddechu**.

```
[[ 1.01449275e+00 -7.22106146e-01 -7.75380188e-01 -8.11778748e-01
 -8.81172614e-02 -7.37541075e-01 -1.29269850e-01  5.78023143e-01
 -7.15918733e-01  5.24679993e-01  6.14196318e-01 -3.07151514e-01
 3.67257932e-01  1.38373991e-01 -8.55973066e-02 -5.98514388e-02]
```

```

3.53569790e-02 -1.31985366e-01  4.19415816e-02 -4.55340894e-01
3.64180002e-01  1.54672420e-01 -2.83404851e-01]
...
[-3.07151514e-01  2.35948706e-01  4.80083310e-01  3.29185366e-01
3.13911413e-01  2.88907792e-02  5.91788533e-01 -3.13289266e-01
4.26860719e-01 -5.42659206e-01 -5.06516346e-03  1.01449275e+00
3.67276804e-01  4.24694604e-04 -1.44316297e-02  2.97652829e-03
-9.28482093e-02  8.58167741e-02  5.12701104e-01 -2.85062961e-01
-6.97949297e-01 -1.19686140e-01  3.37307501e-01]]

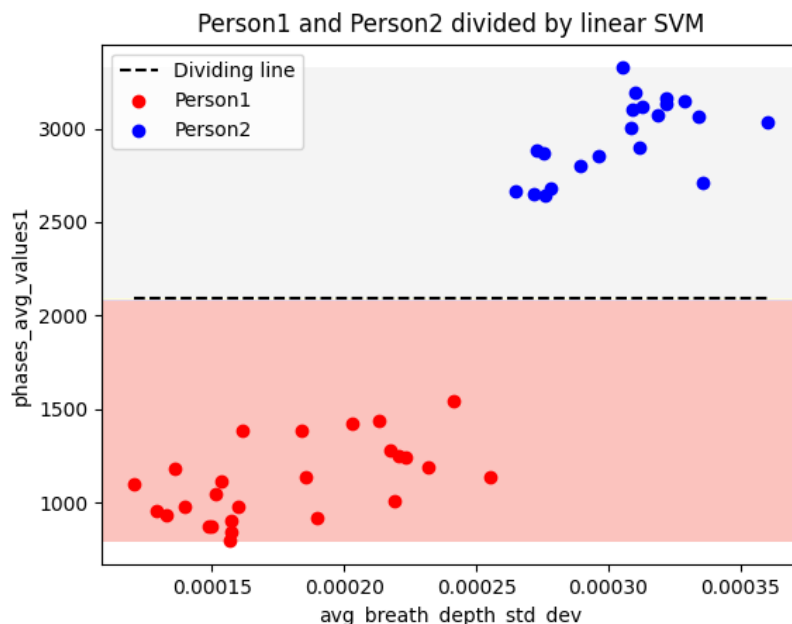
```

Po przygotowaniu przestrzeni cech skrypt wykonuje dwie rzeczy:

- **klasyfikację SVM (ang. *Support Vector Machine*)** - odbywa się ona przy użyciu wytrenowanej (w ramach działania skryptu) pary liniowych wektorów nośnych i użycia ich do rozdzielenia płaszczyzną każdej pary osób należącej do zbioru danych
- **klasyfikację NN (ang. *Neural Network*)** - odbywa się ona przy użyciu wytrenowanej (w ramach działania skryptu) sieci neuronowej, której zadaniem jest klasyfikacja do klasy dla każdej pary osób należącej do zbioru danych

Klasyfikacja SVM

Klasyfikator działa dla dwóch cechy zapisanych w obiekcie `best_pair` efektywnie sprowadzając problem do klasyfikatora binarnego. Trenuje w sposób liniowy `SVC(kernel="linear")`, czyli szuka prostej będącej granicą decyzyjną, która najlepiej rozdziela punkty oby klas. Dla takiej prostej wizualizuje podział jak na wykresie zamieszczonym poniżej oraz wylicza dokładność. Całość odbywa się na tym samym zbiorze danych będącym jednocześnie danymi treningowymi jak i testowymi.



Rysunek 6.4: Wykres przedstawiający binarny klasyfikator SVM rozdzielający dwie klasy

Klasyfikacja NN

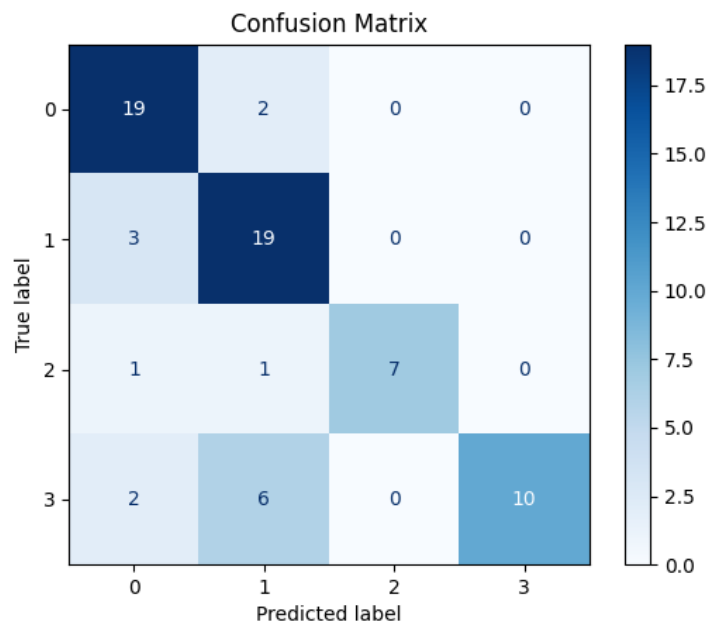
Klasyfikator NN działa w zupełnie inny sposób. Dla tych samych dwóch cech początkowo dokonuje translacji etykiet danych na liczby przy użyciu funkcji `create_person_map()`, a następnie buduje małą sieć neuronową o następującej strukturze:

- wejście: 2 cechy,
- ukryte warstwy: 6 i 10 neuronów,
- wyjście: 4 klasy

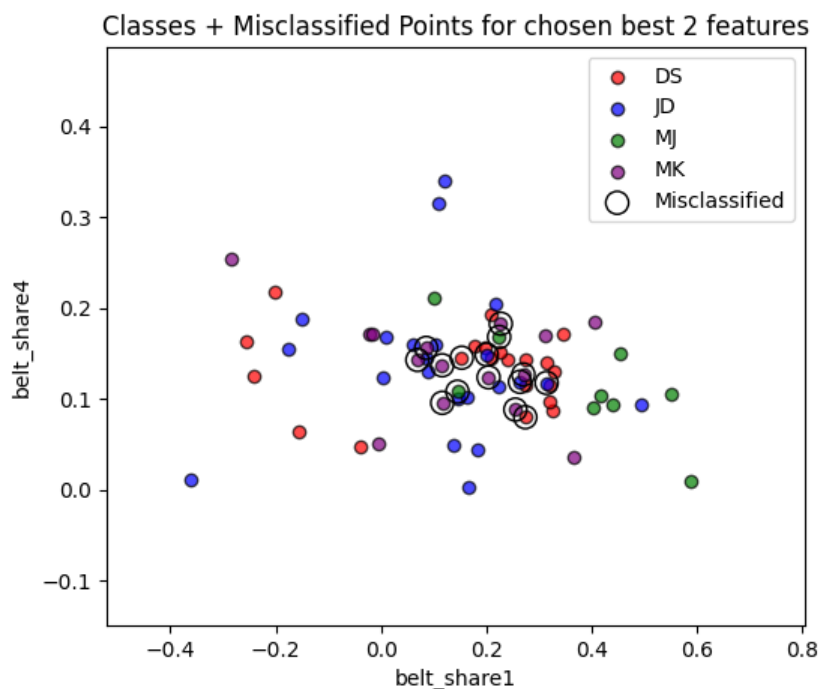
Parametry modelu są następujące:

- funkcja straty: CrossEntropyLoss
- optymalizator: Adam
- stała szybkości uczenia: 0,01

Po treningu dla modelu obliczana jest trafność predykcji oraz tworzona macierz pomyłek, a także wykres przedstawiający klasyfikacje z oznaczonymi pomyłkami. Podobnie jak w przypadku poprzedniego klasyfikatora całość odbywa się na tym samym zbiorze danych będącym jednocześnie danymi treningowymi jak i testowymi.



Rysunek 6.5: Wykres przedstawiający macierz konfuzji klasyfikatora NN dla 4 klas



Rysunek 6.6: Wykres przedstawiający predykcje klas dla 4 klas

Dla powyższych danych i wykresów model osiągnął skuteczność **78,57%**. Widzimy, że dane są skumulowane, co prowadzi do wielu pomyłek na złączeniach cech. Prowadzi to do oczywistego wniosku, że 2 cechy są niewystarczające do podziału więcej niż dwóch osób - wystarczają jako klasyfikator binarny co widać po skuteczności klasyfikatora SVM.

6.3 Dane

Podobnie jak w innych częściach projektu dane zostały podzielone na trzy główne kategorie:

- **Dane surowe(ang. *raw data*)** - dane pozyskane z kamizelki pomiarowej opisane w sekcji 2.1.1.
- **Dane wyczyszczone(ang. *clean data*)** - dane surowe po przejściu przez procedurę data processing opisaną w sekcji 2.2.1
- **Dane cech(ang. *feature data*)** - dane cech wyekstrahowanych z **danych wyczyszczonych** opisane w sekcji 2.2.5.

Dodatkowo struktura folderów została wzbogacona o folder *identifier* zawierający strukturę folderów zgodną z powyżej opisaną, stworzoną do przechowywania plików tymczasowych niezbędnych do poprawnego działania metody `identify_person` klasy `BRICSModelWrapper`.

6.3.1 Pobieranie danych

Ta część projektu została od początku zaprojektowana z myślą o ścisłej współpracy z bazą danych i dostępem do danych poprzez data hooka opisanego w sekcji 3.8. W celu pobrania danych z bazy w użyciu jest mechanizm `download_measurement()`, który zgodnie z zadanym zestawem kryteriów pobiera najnowsze dane dla wszystkich osób (klas) znajdujących się w parametrach zapytania. Następnie wywoływana jest funkcja `load_features_from_database_zip`, której zadaniem jest załadowanie danych cech do odpowiedniego folderu w strukturze projektu. Dane te muszą zostać w odpowiedni sposób oetykietowane - nazwa pliku staje się etykietą i przyjęta zostaje konwencja:

feature_{unikalny identyfikator osoby w bazie danych}.jsonl

Na tak spreparowanych plikach wykonywane sa operacje opisane w sekcjach poniżej.

6.3.2 Parsowanie cech

W powyższej podsekcji wspomniana została procedura zapisu plików wyekstrahowanych cech do odpowiedniej lokalizacji w drzewie projektu. Aby te dane stały się użeteczne należy przygotować dla nich parser - funkcję `parse_features_from_data`. Funkcja ta zmienia dane z plików .jsonl w obiekt klasy `FeatureData` typu `Tensor` z biblioteki *PyTorch*. Dodatkowo na podstawie wierszy nagłówkowych, pozyskuje etykiety do danych, które również trafiają do obiektu. Całość sparsowanych danych oraz etykiet zostaje zapisana do obiektu klasy `ModelData`, który stanowi główny nośnik danych w tej części projektu i opisany został w sekcji 6.4.1.

Dane `feature_id.jsonl` mają następującą postać:

```
{
  {"person_id": "_", "condition": "_", "activity": "_"}
  {"bpm": _, "avg_breath_depth": _, "avg_breath_depth_std_dev": _,
   "phases_avg_values1": _, "phases_avg_values2": _,
   "phases_avg_values3": _, "phases_avg_values4": _,
   "breath_shape1": _, "breath_shape2": _, "breath_shape3": _,
   "breath_shape4": _, "breath_length_variability": _,
   "breath_amplitude_variability": _, "belt_share1": _, "belt_share2": _,
   "belt_share3": _, "belt_share4": _, "belt_share5": _,
   "belt_share_std1": _, "belt_share_std2": _, "belt_share_std3": _,
   "belt_share_std4": _, "belt_share_std5": _}
  ...
  {"bpm": _, "avg_breath_depth": _, "avg_breath_depth_std_dev": _,
   "phases_avg_values1": _, "phases_avg_values2": _,
   "phases_avg_values3": _, "phases_avg_values4": _,
   "breath_shape1": _, "breath_shape2": _, "breath_shape3": _,
   "breath_shape4": _, "breath_length_variability": _,
   "breath_amplitude_variability": _, "belt_share1": _, "belt_share2": _,
   "belt_share3": _, "belt_share4": _, "belt_share5": _,
   "belt_share_std1": _, "belt_share_std2": _, "belt_share_std3": _,
   "belt_share_std4": _, "belt_share_std5": _}
}
```

, gdzie każdy kolejny obiekt *JSON* reprezentuje kolejny dwuminutowy segment danych, z którego te cechy zostały wyznaczone. Po parsowaniu, już w postaci macierzowej wszystkie pliki z cechami (o strukturze przedstawionej powyżej) znajdujące się w folderze `.\data\features` prezentują się następująco:

```
tensor([[ 1.1991, -2.1468, -1.0432, ..., -0.0324,  0.3909,  0.1339],
        [ 0.9716, -0.0349, -0.8515, ..., -0.4965,  0.3156, -0.5367],
        [ 0.9686,  0.0497, -1.0472, ...,  1.8403, -0.1354,  0.3660],
        ...,
        [-0.0128, -0.6737, -0.8988, ...,  0.7098,  0.9672, -0.2827],
        [-0.1824,  0.0486, -0.7419, ..., -0.9169,  0.5260,  2.5993],
        [ 0.0471,  0.2167, -0.8263, ...,  1.2117,  1.4783, -0.9624]])
tensor([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3,
        3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5,
        5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6,
        6, 6, 7, 7, 7, 7, 7, 7, 7, 7, 7, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8])
```


[illegible]

6.4 Model

Głównym celem naszego projektu grupowego było stworzenie modelu nauczania maszynowego, którego celem była identyfikacja osób na podstawie wzorców oddechowych. W związku z tym należało zaplanować technologie i sposób przechowywania informacji o modelu, takich jak: wagi, parametry, słownik do etykiet.

W ramach realizacji sieci neuronowej, zdecydowaliśmy się na użycie biblioteki specjalnie przeznaczonej do tego typu zadań czyli **PyTorch**. Posłuży nam ona nie tylko do trenowania modelu, ale także przygotowywania danych i ewaluacji dokładności modelu.

6.4.1 Klasy będące pojemnikami na dane

Klasy kontenerowe mają na celu uporządkowanie struktury danych wraz z ich metadanymi, nie tylko etykietami, lecz również parametrami takimi jak funkcja straty czy liczba wejść i wyjść. Poniżej przedstawiono je w formie stabularyzowanej:

- Struktura obiektu **FeatureData** - jest to kontener przechowujący dane w formie dwuwymiarowej tablicy, gdzie każdy rząd jest równoznaczny wektorowi cech z jednego segmentu. Do każdego rzędu przypisana jest również etykieta identyfikująca osobę.

Tabela 6.2: Opis struktury obiektu FeatureData.

Pole	Typ	Opis
data	NDArray[float64]	dwuwymiarowa tablica z danymi
labels	NDArray[int]	wektor zawierający etykiety danych

- Struktura obiektu **BRISCSModelWrapper** - jest to klasa umożliwiająca nam przetrzymanie modelu oraz jego parametrów do trenowania czy użytkowania. Dodatkowo są tu również przechowywane słowniki do etykiet oraz cech, przekształcające identyfikatory na łańcuchy znaków. Klasa posiada również metody, które korzystają z wyżej wymienionych słowników, lub z samego modelu.

Tabela 6.3: Opis struktury obiektu BRICSModelWrapper

Pole	Typ	Opis
parameters	dict	Słownik z parametrami treningowymi
people_keys	dict	Słownik mapujący id osoby na osobę
feature_keys	dict	Słownik mapujący id cechy na cechę
model	BRICSModel	Klasa modelu

Metody zaimplementowane w klasie:

Tabela 6.4: Opis metod zaimplementowanych w ramach klasy BRICSModelWrapper.

Metoda	Zwracany typ	Opis
get_person(id: int)	str	Zwraca łańcuch znaków będący żadaną osobą
identify_person(input: Path)	str	Zwraca łańcuch znaków będący osobą zidentyfikowaną
save_model()	None	Zapisuje model i uzupełnia parametry
load_model()	None	Wczytuje model i uzupełnia parametry

- Struktura obiektu **ModelData** - jest to kontener przechowujący poprzednie dwie Klasy oraz obiekty DataLoader potrzebne przy przygotowywaniu kolejnych serii danych do epok.

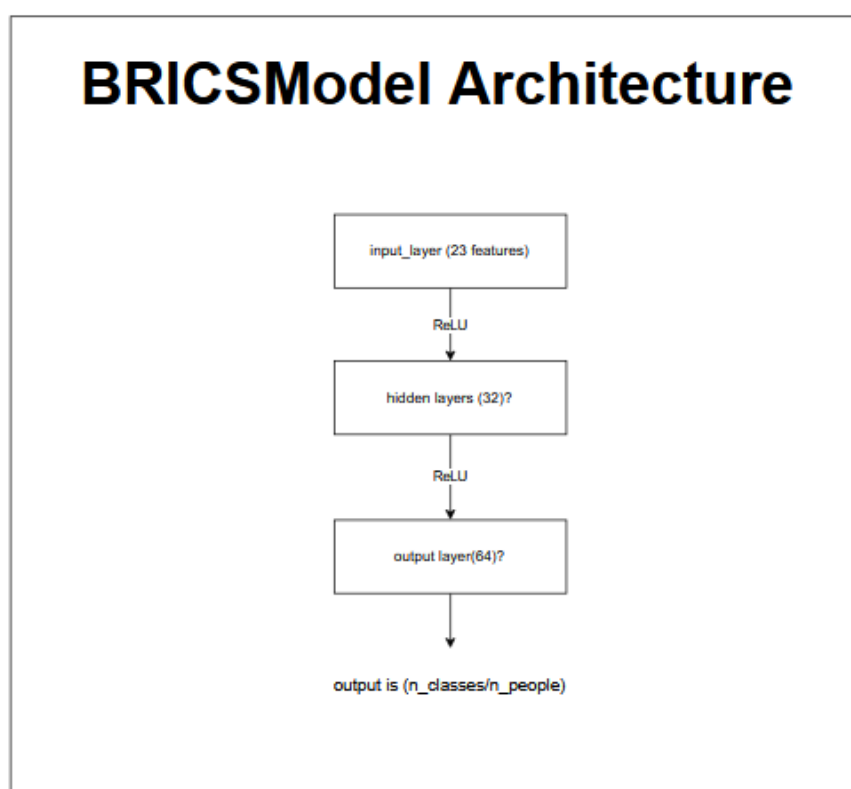
Tabela 6.5: Opis struktury obiektu ModelData.

Pole	Typ	Opis
feature_data	FeatureData	Obiekt z danymi cech
model_wrapper	BRICSModelWrapper	Obiekt z parametrami i modelem
train_loader	DataLoader	Obiekt seryjnego ładowania danych
feature_data	FeatureData	Obiekt seryjnego ładowania danych

6.4.2 Model nauczania maszynowego

Architektura sieci neuronowej

Nasza sieć została stworzona w następującej postaci:



Rysunek 6.7: Diagram przedstawiający strukturę sieci neuronowej. Posiada ona 3 warstwy odpowiadające za przetwarzanie danych:

- **input**
Rozmiarem wejścia dla tej warstwy jest liczba cech w pojedynczym wektorze cech. Dodatkowo, wyjście tej warstwy, w postaci 32 wartości wstępnych cech, jest przekazywane do funkcji aktywacji ReLU przed dalszym przetwarzaniem.
- **hidden**
Liczba wejść do neuronów wejściowych w tej warstwie to 32. Liczba ta jest podyktowana głównie optymalizacją przetwarzania równoległego na rdzeniach karty graficznej, które działają optymalnie dla wartości będących potęgą 2. Dodatkowo, wyjście tej warstwy w postaci 64, bardziej wyszczególnionych cech, również przekazywane jest do funkcji aktywacji ReLU przed dalszym przetwarzaniem.

- **output**

Rozmiar wejścia tej warstwy to 64 i jest to ostatnia warstwa naszego modelu, w której dokonujemy klasyfikacji osób, więc wyjściem będzie wektor wartości z poziomem aktywacji dla każdej klasy(osoby), na której trenowany był model. Wartość ta określana jest mianem logita. Nie są to więc bezpośrednio prawdopodobieństwa klas, co oznacza, że jeżeli chcemy takowe uzyskać, należy wywołać na wyjściu funkcję `softmax`, bądź jeżeli interesuje nas tylko najwyższa wartość, funkcję `argmax`.

6.4.3 Założenia treningowe

W celu wytrenowania naszego modelu musieliśmy poczynić pewne założenia i przygotowywania projektu, tak aby należycie uporządkować naszą przestrzeń pracy. Kluczowe było sformalizowanie poniższych zagadnień:

Sposób przechowywania danych o modelu poza działaniem programu

W celu późniejszego załadowania już wytrenowanego modelu, np. z bazy danych, należało się zastanowić nad formą zapisu parametrów potrzebnych do funkcjonowania. zdecydowaliśmy się więc zapisać 3 poniższe pliki:

- `model.pth` - jest to plik binarny, stworzony przez funkcję zapisu modelu z biblioteki PyTorch. Zawiera on rezultat treningu, czyli zbiór wag neuronów będących rezultatami treningu
- `model.jsonl` - są to parametry zapisane w formacie `jsonl`, które umożliwiają nam załadowanie już istniejącego modelu, ponieważ trzymają one informacje zawarte w polach klas okalających model. Zawartość przykładowego pliku prezentuje się następująco:

```
{
  "parameters": {
    "optimizer": "Adam",
    "criterion": "CrossEntropyLoss",
    "learning_rate": 0.001
  },
  "people_keys": {
    "0": "person1@poczta.pl",
    "1": "person2@gmail.com",
    "2": "person2@onet.pl",
    "3": "person2@mymail.com",
  },
  "feature_keys": {
    "0": "bpm",
    "1": "avg_breath_depth",
    "2": "avg_breath_depth_std_dev",
    "3": "phases_avg_values1",
    "4": "phases_avg_values2",
    "5": "phases_avg_values3",
    "6": "phases_avg_values4",
    "7": "breath_shape1",
    "8": "breath_shape2",
    "9": "breath_shape3",
    "10": "breath_shape4",
    "11": "breath_length_variability",
    "12": "breath_amplitude_variability",
    "13": "belt_share1",
  }
}
```

```

    "14": "belt_share2",
    "15": "belt_share3",
    "16": "belt_share4",
    "17": "belt_share5",
    "18": "belt_share_std1",
    "19": "belt_share_std2",
    "20": "belt_share_std3",
    "21": "belt_share_std4",
    "22": "belt_share_std5"
}
}

```

- **scaler.pkl** - jest to plik przechowujący parametry użyte podczas skalowania danych do treningu. Jest on wymagany w celach późniejszego wczytania go jako obiektu klasy **StandardScaler** odpowiedzialnej za skalowanie danych już w środowisku produkcyjnym, ponieważ tam nie posiadamy danych, na których trenowany był model.

Sposób definiowania hiperparametrów do treningu

W tym celu zdecydowaliśmy się stworzyć osobny plik nazwany **hyperparameters.py**, który zawiera stałe do trenowania takie jak:

- **EPOCHS** - liczba iteracji treningu modelu.
- **BATCH_SIZE** - liczba punktów danych (wektorów cech) ładowanych za jednym przejściem przed wywołaniem propagacji wstecznej.
- **LEARNING_RATE** - stała szybkości uczenia.
- **DROPOUT** - stosunek losowych próbek, które należy odrzucić podczas trenowania w celu dywersyfikacji danych.

Sposób podziału danych treningowych i testowych

Ponieważ nie posiadamy jeszcze bardzo dużej ilości danych, nie zdecydowaliśmy się na zawarcie w naszym procesie uczenia modelu zbioru danych walidacyjnych. W związku z tym zdecydowaliśmy się na najbardziej podstawowy stosunek podziału danych treningowych i testowych **80% danych treningowych, 20% danych testowych**

Ustalenie przepływu danych i przebiegu treningu

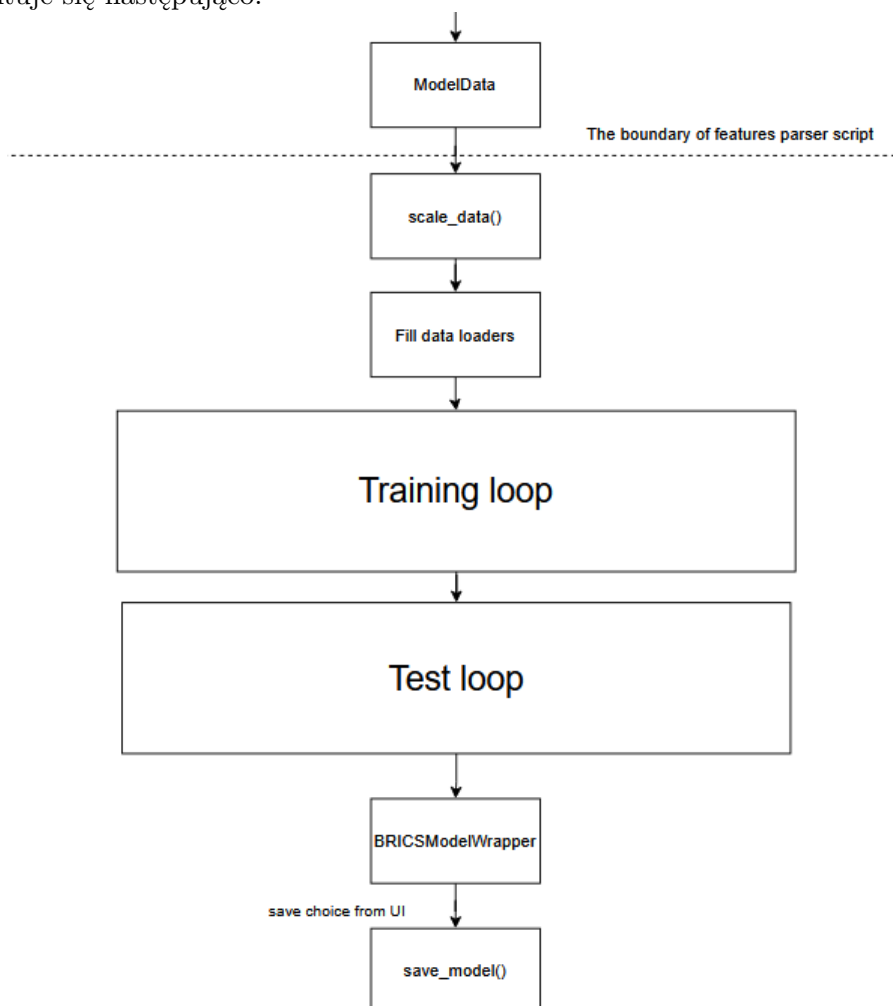
Nasze dane zdecydowaliśmy się umieścić w klasie **DataLoader** z biblioteki PyTorch, gdyż jest to wyjątkowo wygodny sposób ładowania danych w seriach w celu równoleglenia przetwarzania.

Dodatkowo, na tym etapie należało również wybrać nasz optymalizator i funkcję straty, którymi będziemy się posługiwać przy treningu. Zostały więc wybrane dwa sprawdzone rozwiązania, należące do biblioteki:

- optymalizator **Adam** - jest to opcja lepsza niż alternatywa **SGD**, ponieważ dla relatywnie małego zbioru danych osiąga podobne wyniki i cechuje się szybszą zbieżnością. Warto jednak wspomnieć, że system został zaprojektowany w taki sposób, by umożliwiał zmianę tego typu parametrów nawet w czasie uczenia, dzięki wcześniej wspomnianemu słownikowi parametrów **model.jsonl**.
- funkcja straty **CrossEntropyLoss** - jest to podstawowa funkcja występująca w zadaniach klasyfikacji wieloklasowej, którą udostępnia biblioteka.

6.4.4 Przebieg treningu

Trening naszego modelu również został zaplanowany poprzez wspólne wytworzenie diagramu przepływu, który prezentuje się następująco:



Rysunek 6.8: Diagram przepływu przedstawiający procedurę treningu modelu

Na podanym schemacie można zaobserwować dwa główne elementy będące pętlami, kolejno: trenowania i testu z wynikiem w postaci dokładności.

Najważniejsze etapy

To co zostało przedstawione na diagramie, to przepływ danych podczas wywołania skryptu `train_model.py`. w kolejnych etapach realizowane są poszczególne zadania:

1. **scale_data()** - etap opisany został wywołaniem funkcji należącej do skryptu `parse_features.py`, której zadaniem jest przekształcenie załadowanych cech zgodnie z formatem przedstawionym w sekcji data dokumentacji, tak, by model skuteczniej uczył się zależności na danych o podobnym rzędzie wielkości.

Funkcja wywołuje `StandardScaler` z biblioteki PyTorch, który skaluje nasze dane kolumnowo. Oznacza to, że każda kolumna danych, czyli w naszym przypadku wartości danej cechy dla wszystkich pomiarów, jest skalowana zgodnie ze wzorem:

$$\frac{x - \bar{x}}{\sigma}$$

Gdzie:

x to punkt danych

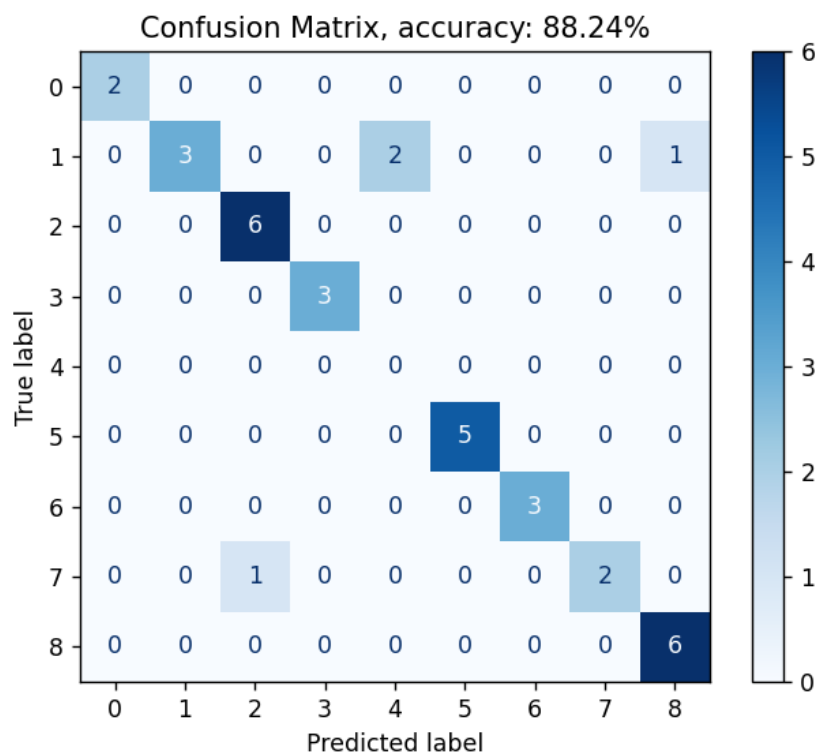
$\bar{x}$ to średnia z kolumny

σ to odchylenie standardowe danych w kolumnie

2. **Fill data loaders** - etap polegający na załadowaniu etykiet, jak i przeskalowanych cech do obiektów `DataLoader` zarówno treningowych jak i testowych. To właśnie tutaj następuje podział zbioru na treningowy i testowy.
3. **Training loop** - etap, w którym przez nasz model przeprowadzamy dane, on zapisuje wynik, liczy funkcję straty i na jej podstawie przeprowadza propagację wsteczną.
4. **Test loop** - etap walidacyjny, gdzie dane z `DataLoader` są analizowane pod kątem poprawności etykiety po przejściu przez model. Wynikiem tej fazy jest dokładność modelu po liczbie epok zdefiniowanej wcześniej. Dodatkowo, tworzona jest również macierz pomyłek dla danych, ale będzie ona przedstawiona szerzej w sekcji aplikacji.
5. **BRICSModelWrapper** - ostatni etap, czyli załadowanie zmian do klasy okalającej model i zapisanie wyników. Po tej fazie występuje również odgórne zapytanie od interfejsu graficznego o zapisanie modelu, jednak nie jest to częścią opisywanego skryptu, a całego przepływu.

6.4.5 Wyniki

Dla tak zadanych parametrów oraz procedury treningu modelu udało się osiągnąć następujące rezultaty:



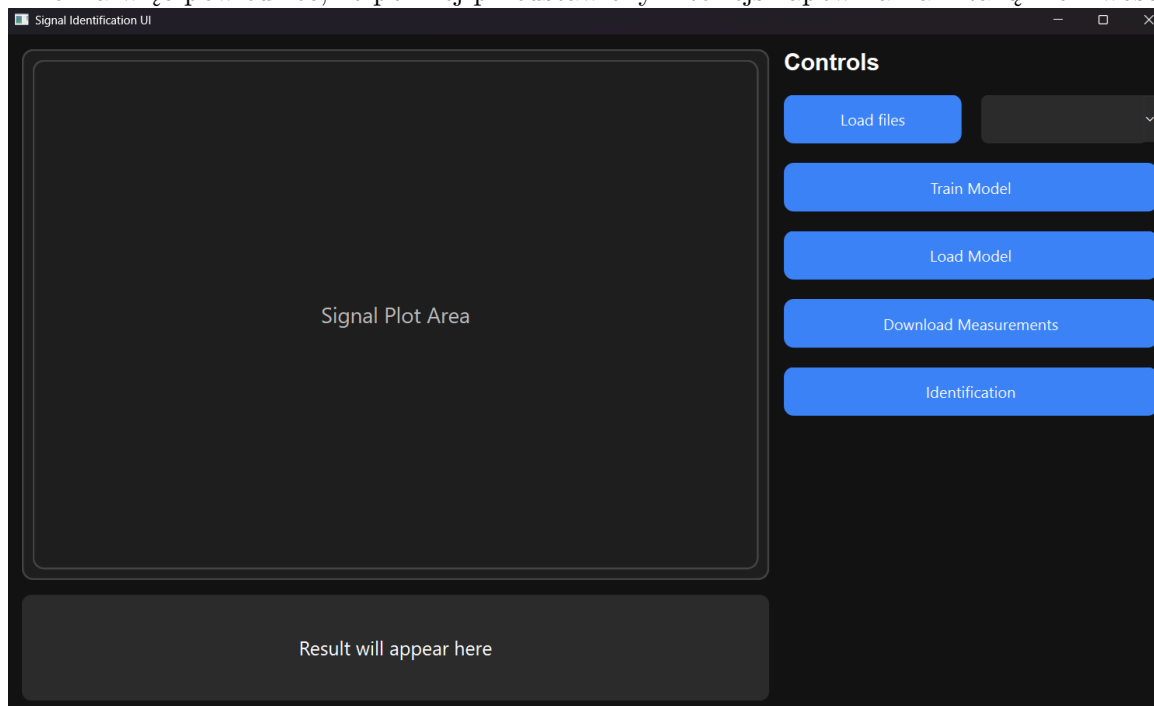
Rysunek 6.9: Wykres przedstawiający wyjściową macierz konfuzji modelu dla dziewięciu klas. Udało się osiągnąć dokładność na poziomie **88,24%**. Na powyższym wykresie jesteśmy w stanie zaobserwować co następuje:

- prawie równomiernie rozłożony zbiór testowy, co może świadczyć o miarodajnej metodzie jego wyznaczania i poprawnym zbiorze danych,
- brak jednej z klas w zbiorze testowym (klasa o numerze 4),
- macierz konfuzji przypominającą macierz diagonalną, co świadczy o wysokiej dokładności modelu.

6.5 Aplikacja

Aplikacja w naszym repozytorium uruchamiana jest poprzez uruchomienie w linii poleceń skryptu `main.py`. Jest to nasz pokazowy produkt i sposób zobrazowania wykonanej pracy nad modelem i przetwarzaniem danych. Interfejs został wykonany w całości za pomocą biblioteki PySide6

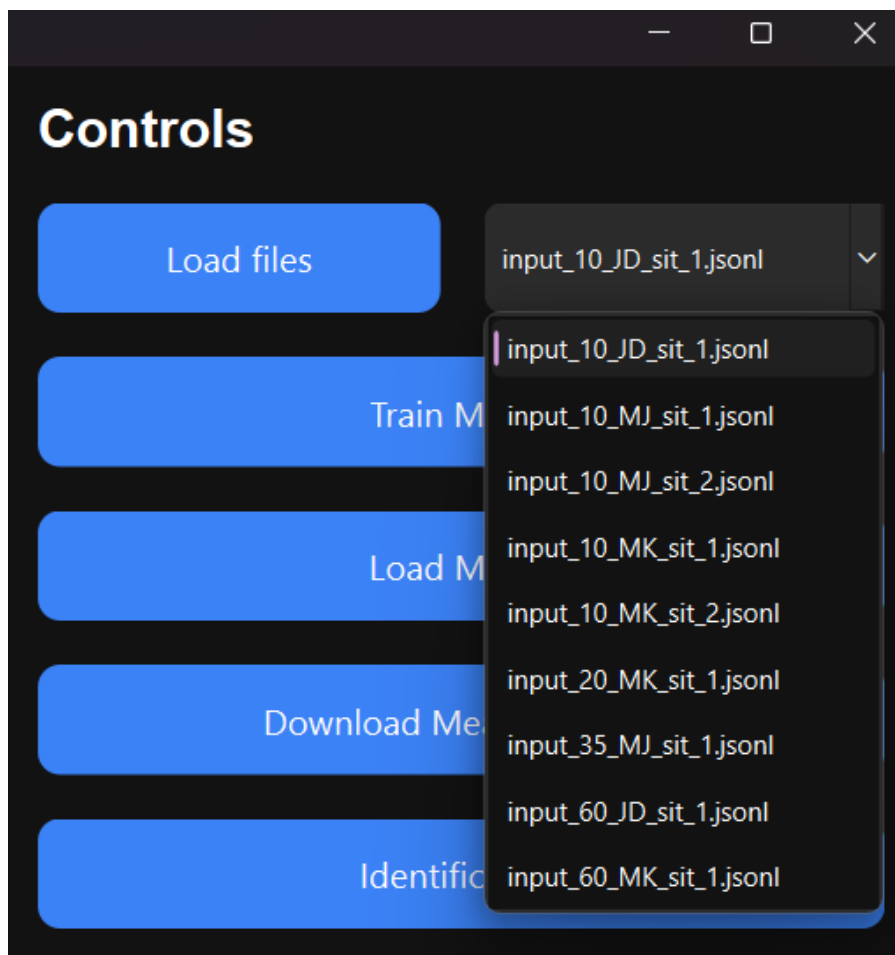
Jednym z naszych założeń projektowych było zgranie wszystkich naszych repozytoriów w jeden spójny system. Można więc powiedzieć, że poniżej przedstawiony interfejs zapewnia nam taką możliwość.



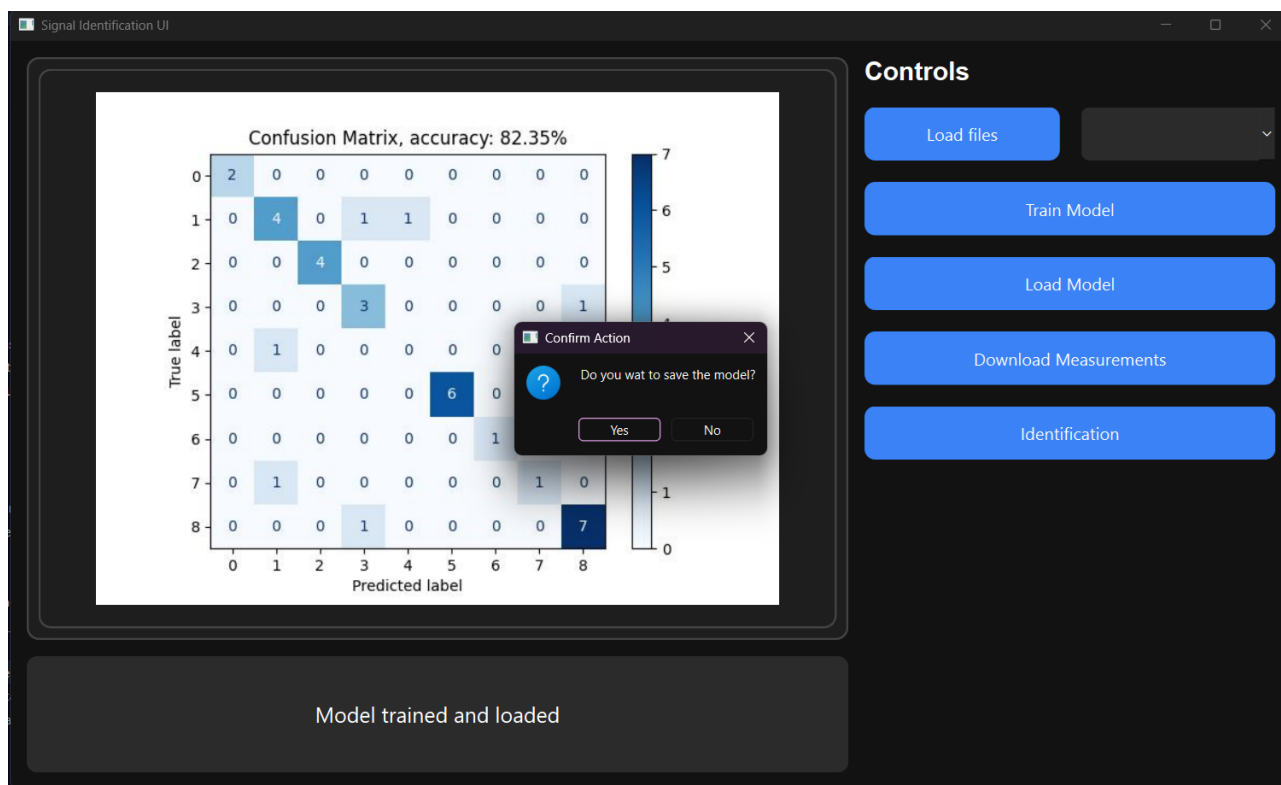
Rysunek 6.10: Zrzut ekranu przedstawiający wygląd ekranu startowego UI modelu
Na przedstawionej grafice można zaobserwować główne funkcje naszego interfejsu:

- **Load files** - przycisk ładujący dane do identyfikacji, tak żeby były możliwe do wyświetlenia w liście rozwijalnej znajdującej się obok.
- **Train Model** - przycisk oferujący możliwość trenowania modelu wraz ze sprzężeniem zwrotnym w postaci macierzy pomyłek danych testowych.
- **Load Model** - przycisk ładujący model do wykorzystania funkcjonalności identyfikacji.
- **Download Measurements** - przycisk oferujący możliwość wywołania skryptu pobierającego pomiary z bazy danych.
- **Identification** - przycisk oferujący naszą główną i docelową funkcjonalność projektową czyli identyfikację osób na podstawie wzorców oddechowych.

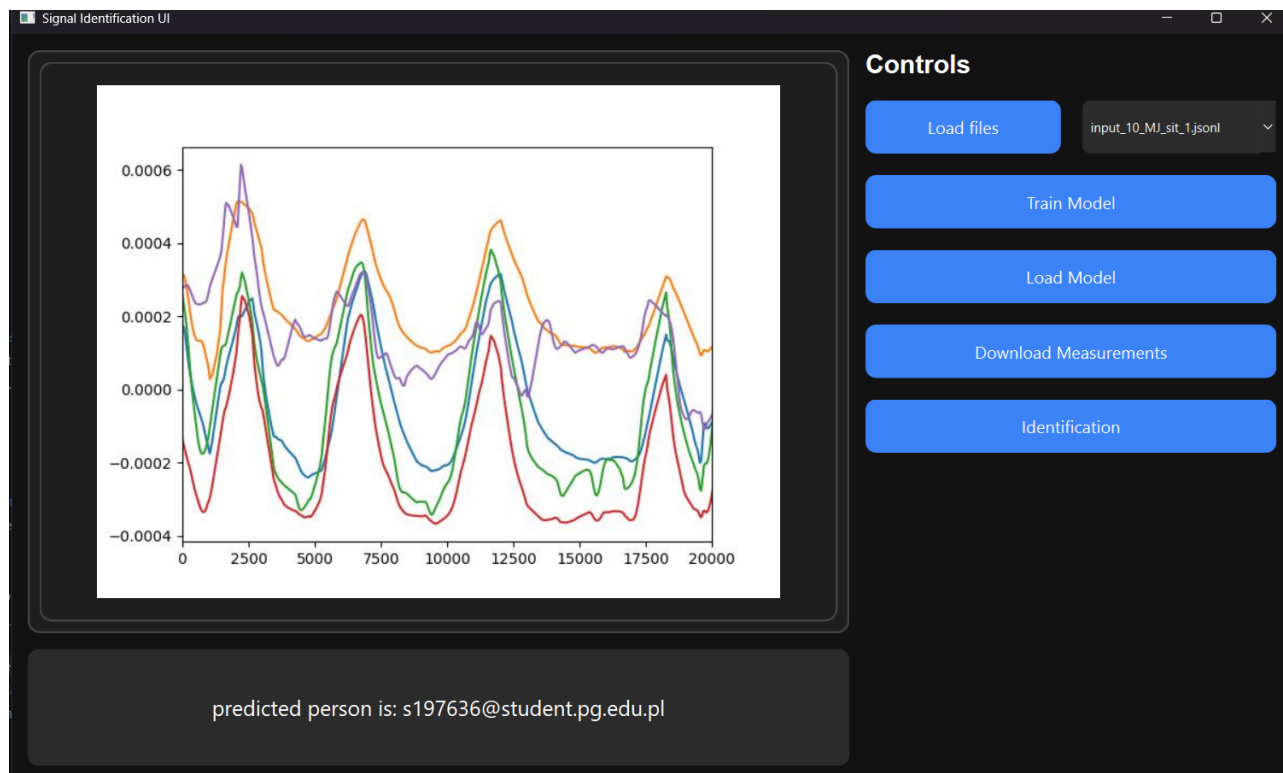
Wyniki podjęcia poszczególnych akcji znajdują się poniżej:



Rysunek 6.11: Interfejs po rozwinięciu listy, po załadowaniu danych



Rysunek 6.12: Interfejs po wywołaniu funkcji trenowania modelu



Rysunek 6.13: Interfejs po wywołaniu funkcji identyfikacji

Spis rysunków

1.1	Graficzna reprezentacja sposobu noszenia kamizelki pomiarowej BRV.	5
2.1	Wykryte anomalie na podstawie amplitudy dla przykładowego sygnału.	10
2.2	Wykryte anomalie na podstawie Częstotliwości dla przykładowego sygnału.	10
2.3	Wykryte anomalie po usunięciu artefaktów dla przykładowego sygnału.	11
2.4	Odtworzony sygnał po usunięciu anomalii dla przykładowego sygnału.	11
2.5	Cechy po wywołaniu algorytmu MDS przedstawione na dwuwymiarowym wykresie. . .	14
2.6	Cechy po wywołaniu algorytmu PCA przedstawione na dwuwymiarowym wykresie. . .	14
2.7	Wykres przedstawiający podział klasyfikatorem SVM dla dwóch klas	15
2.8	Wykres przedstawiający podział klasyfikatorem SVM dla dwóch klas	15
2.9	Wykres przedstawiający podział klasyfikatorem SVM dla dwóch klas	16
2.10	Przykładowy cykl oddechowy stanowiący najbardziej reprezentatywną próbkę.	16
2.11	Przykładowa reprezentacja graficzna profilu oddechowego w postaci wygenerowanego syntetycznego sygnału.	17
2.12	Przykładowa reprezentacja graficzna profilu oddechowego w postaci wybranego rzeczywistego cyklu oddechowego.	17
2.13	Przykładowa reprezentacja sygnału oddechowego wygenerowanego w oparciu o powyższy opis.	18
3.1	Diagram przedstawiający przepływ danych w potoku <i>data_processing_pipeline</i>	19
4.1	Przykładowy rekord metadanych pomiaru	25
4.2	Uproszczony diagram transmisji danych wewnątrz projektu (pomijając kamizelkę BRV)	28
6.1	Diagram przedstawiający strukturę pakietów w repozytorium BRICSModel	33
6.2	Wykres przedstawiający macierz konfuzji klasyfikatora danych oczyszczonych dla 4 klas	35
6.3	Wykres przedstawiający macierz konfuzji klasyfikatora danych surowych dla 4 klas . .	36
6.4	Wykres przedstawiający binarny klasyfikator SVM rozdzielający dwie klasy	37
6.5	Wykres przedstawiający macierz konfuzji klasyfikatora NN dla 4 klas	38
6.6	Wykres przedstawiający predykcje klas dla 4 klas	39
6.7	Diagram przedstawiający strukturę sieci neuronowej	42
6.8	Diagram przepływu przedstawiający procedurę treningu modelu	45
6.9	Wykres przedstawiający wyjściową macierz konfuzji modelu dla dziewięciu klas	46
6.10	Zrzut ekranu przedstawiający wygląd ekranu startowego UI modelu	47
6.11	Interfejs po rozwinięciu listy, po załadowaniu danych	48
6.12	Interfejs po wywołaniu funkcji trenowania modelu	48
6.13	Interfejs po wywołaniu funkcji identyfikacji	49

Spis tabel

2.1	Opis składowych danych surowych.	7
3.1	Opis struktury klasy BRVDataIntermediate.	19
3.2	Opis struktury klasy BRVDataIntermediate.	19
4.1	Opis struktury obiektu MeasurementMetadata	24
4.2	Opis struktury klasy LabelsData	24
4.3	Opis struktury klasy BioData	24
4.4	Opis struktury klasy ModelMetadata	25
6.1	Reprezentacja wyników zwracanych przez klasyfikator danych oczyszczonych.	35
6.2	Opis struktury obiektu FeatureData.	41
6.3	Opis struktury obiektu BRICSModelWrapper	41
6.4	Opis metod zaimplementowanych w ramach klasy BRICSModelWrapper.	41
6.5	Opis struktury obiektu ModelData.	42

Bibliografia

- [1] A. LoMauro, A. Colli, L. Colombo, and A. Aliverti, *Breathing patterns recognition: A functional data analysis approach*, Computer Methods and Programs in Biomedicine, vol. 221, 2022.
- [2] dr hab. inż. Julian Szymański, *Respiratory Rhythm Phases Classification Dataset* , 2025
- [3] Zelano Lab, Torben Noto, *BreathMetrics*, 2019